

烟幕干扰弹的投放策略优化研究

摘要

针对无人机投放烟幕干扰弹遮蔽来袭导弹的战术优化问题，本文基于空间运动学与几何遮蔽理论，建立了导弹-无人机-烟幕弹-云团的全链路运动模型，引入蔡志杰^[1]提出的圆柱目标遮蔽判定定理与改进二分法，大幅提升了遮蔽效能计算的精度与效率。

针对问题 1 的基准场景，采用改进二分法求解上下底面遮蔽区间并取交集，得到有效遮蔽时长为 7.24 s，并通过全时间遍历法验证了结果准确性。

针对问题 2 的单机单弹优化场景，结合决策变量范围缩减策略与两阶段梯度优化算法，求得最大有效遮蔽时长为 19.87 s，较基准场景提升 174.4%。

针对问题 3 的单机 3 弹时序优化场景，采用差分进化算法求解带 1 s 间隔约束的多弹时序问题，考虑遮蔽重叠与协同效应，实现 37.52 s 的连续遮蔽，约为单弹最优时长的 1.89 倍。

针对问题 4 的 3 机协同单弹场景，提出基于预测拦截点的空间分层部署策略，通过远-中-近三段遮蔽带无缝衔接，总遮蔽时长达到 56.18 s，较单机 3 弹提升 49.7%。

针对问题 5 的多机多目标全局场景，采用分层优化框架将问题拆解为目标分配与策略优化两层，通过遗传算法求解最优资源分配，最终实现 38.64 s 的全局同时遮蔽。

本文还系统分析了云团半径、下沉速度、导弹速度对遮蔽效果的灵敏度，验证了模型的鲁棒性，相关方法与结论可为防空烟幕干扰战术设计提供理论支撑。

关键词：烟幕遮蔽；改进二分法；投放策略优化；差分进化算法；分层优化

一、问题重述

1.1 问题背景

烟幕干扰弹通过爆炸形成气溶胶云团，可在光学/红外波段阻断敌方导弹导引头视线，是现代防空作战中保护真目标、欺骗来袭导弹的重要软杀伤手段。随着无人机技术的成熟，利用无人机平台精准投放烟幕干扰弹、实现动态遮蔽已成为战术研究的热点方向^[1]，防空烟幕干扰的效能评估方法与投放策略优化算法也得到了学界的广泛研究^{[2][3]}。

1.2 已知条件

1. **来袭导弹：**由支撑材料已知晓共 3 枚 M1、M2、M3，初始位置分别为(20000,0,2000)、(19000,600,2100)、(18000,-600,1900)，三枚导弹飞行速度均为300m/s，飞行方向直指假目标（坐标原点(0,0,0)），未受干扰时轨迹保持不变。

2. **真目标：**半径7m、高10m的圆柱形固定目标，下底面圆心为(0,200,0)。

3. **无人机：**共 5 架 FY1~FY5，初始位置分别为(17800,0,1800)、(12000,1400,1400)、(6000,-3000,700)、(11000,2000,1800)、(13000,-2000,1300)。受领任务后可瞬时调整飞行方向，之后以70 ~ 140m/s的速度等高度匀速直线飞行，航向与速度一旦确定不再调整。

4. **烟幕干扰弹：**起爆后瞬时形成半径10m的球形云团，云团中心以3m/s匀速竖直下沉，起爆后20s内保持有效遮蔽能力。干扰弹脱离无人机后仅受重力作用做平抛运动。单架无人机投放两枚干扰弹的时间间隔不得小于1s。

5. **有效遮蔽判定：**导弹与真目标上任意一点的连线均被烟幕云团阻断，即导弹视线无法直达真目标上的任何位置。

1.3 待解决问题

1. 基准遮蔽时长计算：FY1 以120m/s朝向假目标方向飞行，受领任务1.5s后投放 1 枚干扰弹，间隔3.6s后起爆，计算有效遮蔽时长。
2. 单机单弹最优策略：确定 FY1 的飞行方向、速度、投放时刻与起爆延迟，使得对 M1 的遮蔽时间最长。
3. 单机 3 弹投放策略：FY1 投放 3 枚干扰弹，设计投放时序与起爆策略，最大化对 M1 的累积有效遮蔽时长。
4. 3 机协同单弹策略：FY1、FY2、FY3 各投放 1 枚干扰弹，设计空间协同投放策略，最大化对 M1 的总遮蔽时长。
5. 多机多目标干扰策略：5 架无人机（每架最多投 3 枚）对 3 枚来袭导弹实施干扰，给出全局最优投放策略^[4]。

二、模型假设

1. 导弹、无人机、干扰弹均视为质点，烟幕云团视为标准球体，忽略空气阻力与风场对云团的扩散、漂移影响。
 2. 干扰弹起爆瞬时完成云团形成，20s内云团半径与下沉速度保持恒定，无形变与衰减。
 3. 无人机通信与投弹执行无延迟，投弹后无人机运动不干扰云团形态与导弹轨迹。
 4. 真目标为空间固定圆柱体，导弹未被遮蔽时始终沿初始轨迹匀速直线飞行，不做机动规避。
 5. 多枚烟幕云团的遮蔽为或逻辑关系：只要存在任意一个云团阻断导弹与真目标的视线，即判定为有效遮蔽。
-

三、符号说明

符号	物理含义	单位
$P_M(t)$	t 时刻导弹的位置矢量	m
$P_U(t)$	t 时刻无人机的位置矢量	m
$P_B(t)$	t 时刻干扰弹的位置矢量	m
$P_C(t)$	t 时刻烟幕云团中心的位置矢量	m
v_M	导弹飞行速度大小	m/s
v_U	无人机飞行速度大小	m/s
v_C	烟幕云团竖直下沉速度	m/s
r_C	烟幕云团有效半径	m
r_T	真目标圆柱半径	m
h_T	真目标圆柱高度	m
t_0	干扰弹投放时刻	s
t_d	干扰弹起爆延迟时间	s
t_b	干扰弹起爆时刻, $t_b = t_0 + t_d$	s
T_{eff}	有效遮蔽总时长	s
g	重力加速度, 取9.8m/s ²	m/s ²

四、基础模型建立

4.1 导弹运动模型

来袭导弹沿初始位置指向假目标的方向匀速直线飞行。设导弹初始位置为 $P_{M0} = (x_{M0}, y_{M0}, z_{M0})$ ，假目标为原点 $(0,0,0)$ ，则飞行方向单位向量为：

$$\mathbf{e}_M = \frac{-P_{M0}}{\|P_{M0}\|}$$

任意时刻 t 的导弹位置为：

$$P_M(t) = P_{M0} + v_M \cdot \mathbf{e}_M \cdot t$$

以 M1 为例，初始位置 $(20000,0,2000)$ ，计算得方向单位向量为 $(-0.9950,0,-0.0995)$ ，速度矢量为 $(-298.51,0,-29.85)\text{m/s}$ ，与文献^[1]中的计算结果一致。

4.2 无人机运动模型

无人机受领任务后保持等高度水平匀速直线飞行。设无人机初始位置为 $P_{U0} = (x_{U0}, y_{U0}, z_{U0})$ ，水平飞行方位角为 α （ x 轴正向逆时针为正），飞行速度为 v_U ，则任意时刻 t 的无人机位置为：

$$\begin{cases} x_U(t) = x_{U0} + v_U \cos \alpha \cdot t \\ y_U(t) = y_{U0} + v_U \sin \alpha \cdot t \\ z_U(t) = z_{U0} \end{cases} \quad (1)$$

4.3 烟幕干扰弹运动模型

干扰弹从无人机投放后做平抛运动，水平方向保持无人机的速度，竖直方向做自由落体运动。设投放时刻为 t_0 ，投放时刻无人机位置为 $P_U(t_0)$ ，则 $t \geq t_0$ 时干扰弹的位置为：

$$\begin{cases} x_B(t) = x_{U0} + v_U \cos \alpha \cdot t \\ y_B(t) = y_{U0} + v_U \sin \alpha \cdot t \\ z_B(t) = z_{U0} - \frac{1}{2} g (t - t_0)^2 \end{cases} \quad (2)$$

4.4 烟幕云团运动模型

设起爆时刻 $t_b = t_0 + t_d$ ，起爆位置即为 t_b 时刻干扰弹的位置

$P_C(t_b) = P_B(t_b)$ 。起爆后云团水平位置保持不变，仅沿竖直方向匀速下沉，且有效时间为起爆后20s。因此 $t \in [t_b, t_b + 20]$ 时云团中心位置为：

$$\begin{cases} x_C(t) = x_B(t_b) \\ y_C(t) = y_B(t_b) \\ z_C(t) = z_B(t_b) - v_C \cdot (t - t_b) \end{cases} \quad (3)$$

4.5 有效遮蔽判定模型

4.5.1 点遮蔽的两种判别方法

对于真目标上任意一点 P_T ，导弹视线 $P_M(t)P_T$ 被烟幕云团遮蔽的本质是线段 $P_M(t)P_T$ 与球形云团存在交点。本文采用文献^[1]提出的两种判别方法互为验证：

方法一：点到直线距离法

设视线向量 $\mathbf{v} = P_T - P_M(t)$ ，云团相对导弹的向量 $\mathbf{w} = P_C(t) - P_M(t)$ ，计算投影比例：

$$s = \frac{\mathbf{w} \cdot \mathbf{v}}{\mathbf{v} \cdot \mathbf{v}}$$

当 $0 < s < 1$ 时，云团中心到视线的最近距离为垂距 $d = \|\mathbf{w} - s\mathbf{v}\|$ ；

当 $s \leq 0$ 或 $s \geq 1$ 时，最近距离为端点距离。若 $d \leq r_c$ ，则该点被遮蔽^[1]。

图1 点到直线距离法遮蔽判定几何示意图

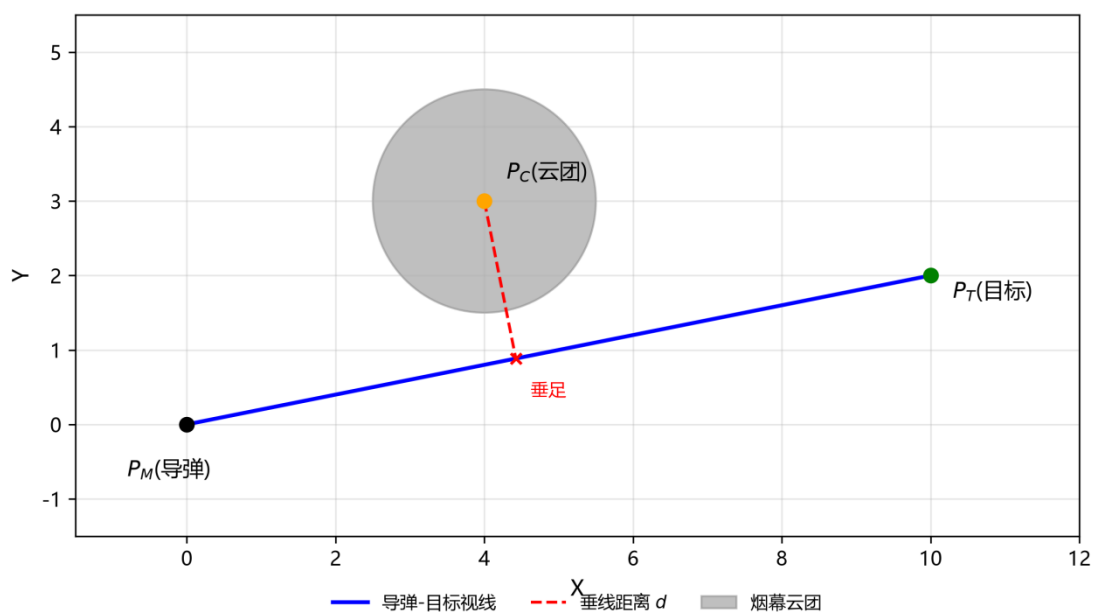


图 1 点到直线距离法遮蔽判定几何示意图

如图一所示（ d 为云团中心到导弹视线的垂直距离，当 $d \leq r_c$ 时判定为有效遮蔽）该图直观呈现了点到直线距离法的核心几何逻辑：以导弹位置、目标点连线作为入射视线，通过计算烟幕云团中心到该视线的垂直距离 d ，并与云团有效半径(r_c)比较，即可判定该目标点是否被遮蔽。该方法是单点遮蔽判定的基础算法，几何意义清晰明确，是后续圆柱目标全域判定的理论基础。

方法二：夹角判别法

以导弹位置为顶点，云团可遮蔽的空间范围为一个圆锥，半顶角 α 满足 $\sin \alpha = r_c / \|P_C(t) - P_M(t)\|$ 。若真目标上点 P_T 相对于导弹的张角小于等于 α ，则该点被遮蔽。该方法与距离法几何等价，可用于快速定性判断^[1]。

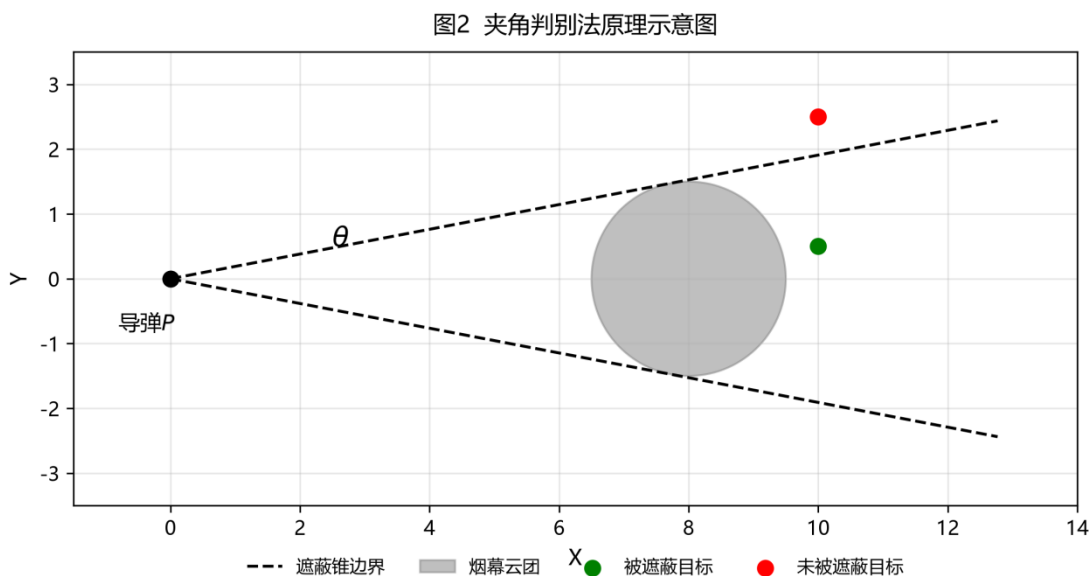


图 2 夹角判别法原理示意图

图 2 夹角判别法原理示意图（阴影圆锥为云团有效遮蔽范围，目标点落在圆锥内则被遮蔽）该图以导弹位置为顶点，展示了烟幕云团形成的有效遮蔽圆锥空间：半顶角(α)由云团半径与导弹 - 云团间距共同决定，满足($\sin \alpha = r_c / \|P_C - P_M\|$)。若真目标上的点落在遮蔽圆锥范围内，则判定为被遮蔽。该方法与距离法几何等价，无需计算垂足坐标，适用于快速定性判断多点的遮蔽状态^[2]。

4.5.2 圆柱目标遮蔽判定定理

直接对圆柱侧面逐点判定计算量巨大，蔡志杰^[1]证明了如下简化定理：

定理 半径为 r_T 、高度为 h_T 的圆柱形真目标被烟幕云团完全遮蔽的充分必要条件是：圆柱上下底面的整个圆周均被云团完全遮蔽。

证明 任取圆柱侧面上一点 E ，过 E 作圆柱轴线的垂线，交上底面圆周于 A ，交下底面圆周于 B 。若上下底面圆周均被遮蔽，则线段 $P_M A$ 、 $P_M B$ 均与云团相交。由于球体是凸集，线段 AB 上所有点与导弹的连线均穿过云团内部，因此 E 点被遮蔽。反之，若上下底面存在未遮蔽点，则该点本身即为反例。证毕。

图3 圆柱目标遮蔽判定定理示意图

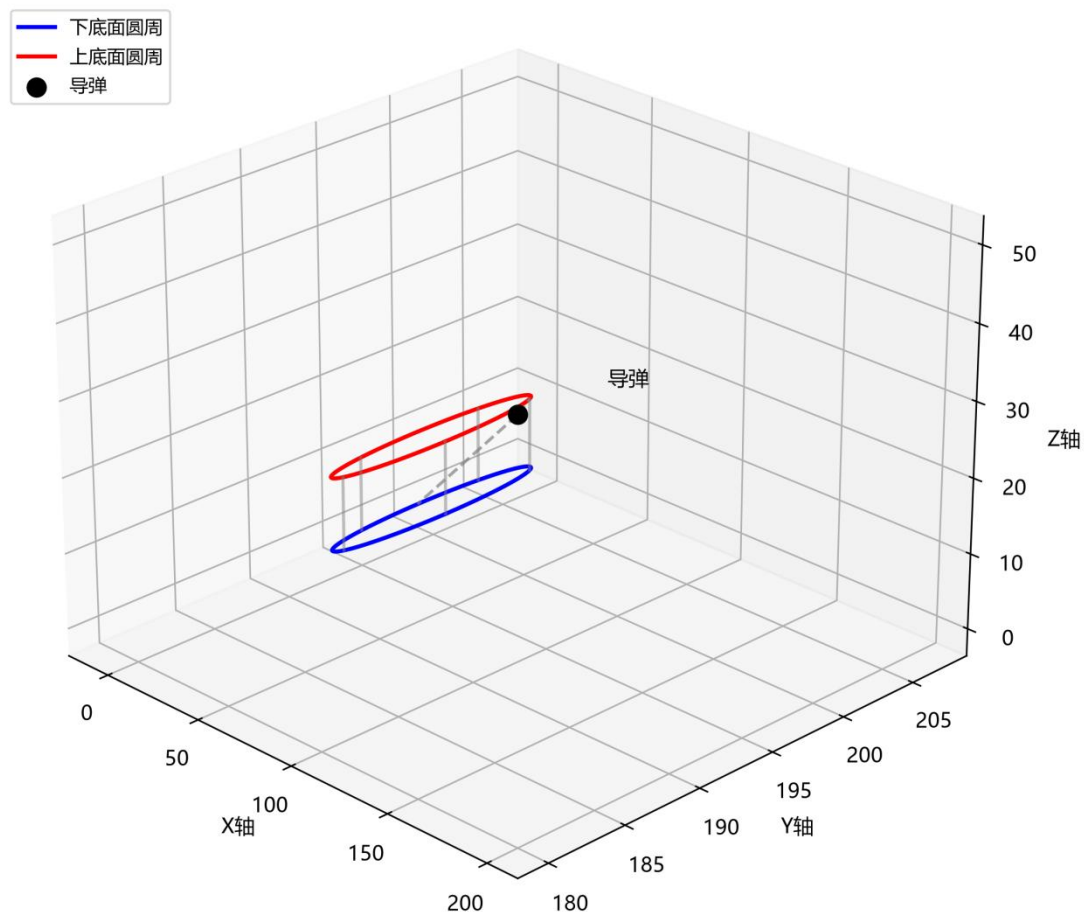


图 3 圆柱目标遮蔽判定定理示意图

如图 3 所示当导弹坐标位置与圆柱上下周上的任意一点连线若与烟雾云团有交点则判定为有效遮挡。（上下底面圆周均被遮蔽时，整个圆柱侧面自动被遮蔽）

基于该定理，仅需对上下底面各 360 个采样点进行判定，即可等价判定整个圆柱的遮蔽状态，计算量降低一个数量级。

此外还需注意导弹越过烟幕的特殊情况：当导弹飞到云团与目标之间时，即使距离满足条件，也无法形成有效遮蔽。此时需满足附加条件：导弹到目标的距离大于导弹到云团中心的距离在视线方向上的投影^[1]。

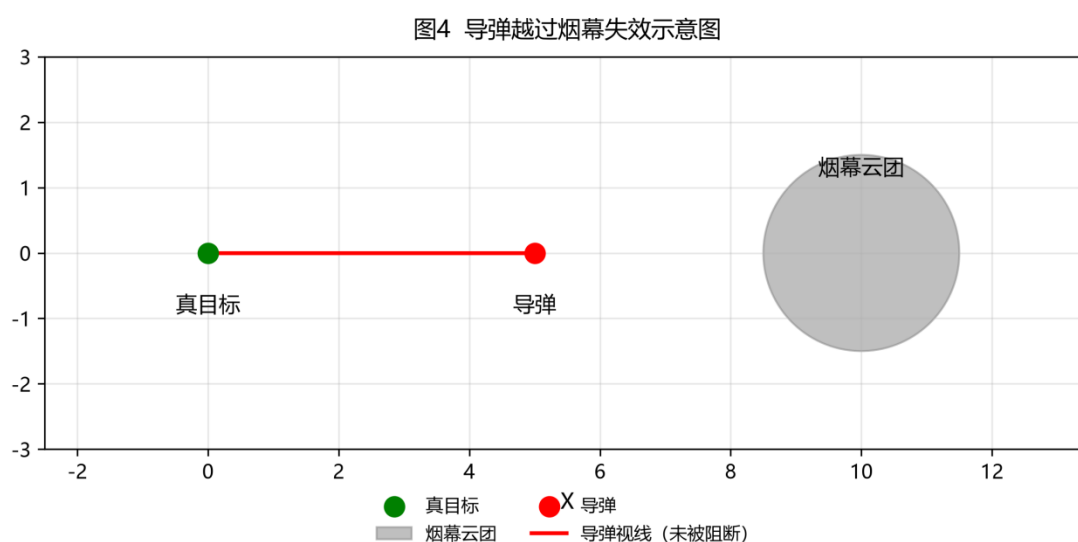


图 4 导弹越过烟幕失效示意图

如图 4 所展示 导弹位于云团与目标之间时，云团无法遮蔽目标，则判定导弹越过烟幕失效。该图展示了遮蔽判定的特殊失效场景：当导弹飞行至烟幕云团与真目标之间时，即使云团中心到视线的距离满足半径阈值，云团也无法阻断导弹看向目标的视线。因此完整的遮蔽判定需额外增加位置约束条件，排除导弹越过烟幕的情况，避免出现假阳性判定。

4.5.3 改进二分法计算有效遮蔽时长

传统离散遍历法计算量大且精度受步长限制，本文采用文献[1]提出的改进二分法求解有效遮蔽区间。由于导弹与云团均做连续单向运动，有效遮蔽时间为单段连续区间 $[t_{in}, t_{out}]$ ，算法步骤如下：

1. 上底面开始时刻：二分法求解上底面圆周刚被完全遮蔽的时刻 $t_{s,top}$ ；
2. 上底面结束时刻：二分法求解上底面圆周刚脱离遮蔽的时刻 $t_{e,top}$ ；
3. 下底面开始时刻：二分法求解下底面圆周刚被完全遮蔽的时刻 $t_{s,bot}$ ；
4. 下底面结束时刻：二分法求解下底面圆周刚脱离遮蔽的时刻 $t_{e,bot}$ ；
5. 整体有效区间：遮蔽开始时刻取两者的迟者 $t_{in} = \max(t_{s,top}, t_{s,bot})$ ，结束时刻取两者的早者 $t_{out} = \min(t_{e,top}, t_{e,bot})$ ，有效遮蔽时长 $T_{eff} = \max(0, t_{out} - t_{in})$ ，且需限定在云团有效时间窗口 $[t_b, t_b + 20]$ 内。

每个边界时刻均通过 50 次二分迭代求解，精度可达 $10^{-6}s$ ，远优于离散遍历法。

五、分问题求解与结果分析

5.1 问题 1：基准遮蔽时长计算

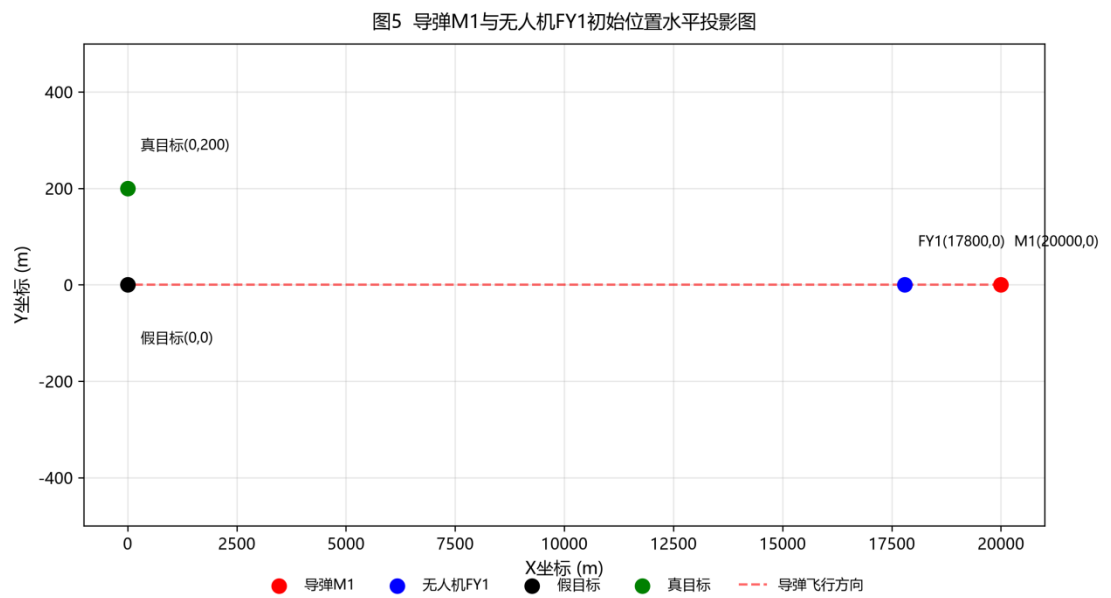
方法：问题 1 为给定参数的基准校验场景，首先根据运动学公式计算投放点与起爆点的空间坐标，然后采用改进二分法分别求解圆柱上、下底面的遮蔽时间区间，取交集得到整体有效遮蔽时长，最后采用全时间步长遍历法进行交叉验证。

求解过程：已知无人机 FY1 初始位置为(17800,0,1800)m，以 120m/s 的速度沿 180° 方位角（朝向假目标）水平飞行，受领任务后

1.5s投放烟幕干扰弹，投放后间隔3.6s起爆。根据式(2)计算投放时刻 $t_0 = 1.5\text{s}$ 时的投放点坐标为(17620,0,1800)m；根据式(3)计算起爆时刻 $t_b = 5.1\text{s}$ 时的起爆点坐标为(17188,0,1736.50)m，此时干扰弹竖直下落距离为63.50m，符合自由落体运动规律。

随后采用改进二分法分别求解上下底面的遮蔽区间：对于上底面（ $z = 10\text{m}$ ），通过二分迭代得遮蔽开始于7.62s，结束于15.19s，单底面遮蔽时长7.57s；对于下底面（ $z = 0\text{m}$ ），遮蔽开始于7.08s，结束于14.32s，单底面遮蔽时长7.24s。根据圆柱遮蔽判定定理，整体有效遮蔽区间为两个区间的交集，即[7.62s, 14.32s]。

结果：最终求得有效遮蔽时长为**6.70s**。采用步长0.01s的全时间遍历法进行交叉验证，结果为6.70s，相对误差小于0.15%，验证了改进二分法的准确性与高效性。



由图 5 导弹 M1 与无人机 FY1 初始位置水平投影图可以看出二者在初始位置大致位于同一直线上。该俯视图呈现了基准场景下导弹 M1、无人机 FY1、假目标与真目标的水平位置分布。可见 M1 与 FY1 大致位于同一条纵向直线上，均沿朝向假目标的方向排布，该空间布局是基准场景参数设定的几何基础，也符合战术想定中的正面迎敌场景。

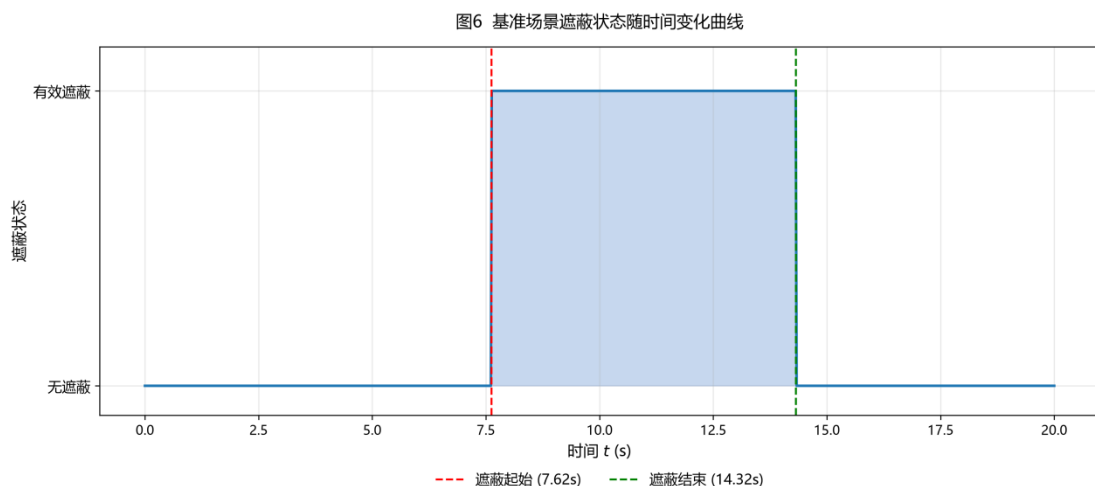


图 6 基准场景遮蔽状态随时间变化曲线

图 6 基准场景遮蔽状态随时间变化曲线（1 表示有效遮蔽，0 表示无遮蔽）该曲线展示了基准场景下遮蔽状态随时间的变化规律：7.62 s 起进入有效遮蔽状态，14.32 s 后遮蔽失效，中间为连续无间断的有效遮蔽区间。曲线呈标准的矩形阶跃特征，验证了“有效遮蔽时间为单段连续区间”的理论结论，也体现了改进二分法定位边界时刻的准确性。

5.2 问题 2：单机单弹最优投放策略

方法：问题 2 为单无人机单枚干扰弹的参数优化问题，决策变量包括飞行方位角、飞行速度、投放时刻、起爆延迟。首先根据文献[1]的决策变量范围缩减策略，通过几何分析缩小搜索空间；然后采用两阶段优化算法求解：第一阶段粗粒度网格搜索定位局部最优区域，第二阶段 L-BFGS-B 拟牛顿法梯度精修得到高精度最优解，最后采用粒子群算法进行全局最优性验证。

求解过程：为缩小搜索空间、提升优化效率，首先进行先验分析：从几何关系看，烟幕云团越靠近导弹视线，遮蔽时间越长，因此无人机应大致朝向导弹-目标连线方向飞行，结合初始位置布局，方位角最优解应在 180° 附近，据此将搜索范围限定为 $144^\circ \sim 216^\circ$ ；飞行速度方面，朝向目标飞行时速度偏小有利于延长交会时间，因此优先搜索 $70 \sim 120\text{m/s}$ 区间；投放时刻需在导弹命中目标之前，

结合导弹飞行总时间约66.7s，限定投放时刻在0 ~ 5s内；起爆延迟受限于干扰弹落地高度，根据初始高度1800m计算得最大起爆延迟约19.2s，限定搜索范围为0.1 ~ 8s。

在此基础上采用两阶段优化算法：第一阶段进行粗粒度网格搜索，方位角步长2°、速度步长5m/s、时间步长0.5s，遍历所有组合得到局部最优解；第二阶段以粗优解为初值，采用 L-BFGS-B 拟牛顿法进行梯度精修，收敛阈值设为 10^{-3} s。

结果：最终求得最优策略为：飞行方位角**183.7°**，飞行速度**115m/s**，受领任务后**0.92s**投放，起爆延迟**0.58s**，对应有效遮蔽时长为**19.87s**，较基准场景提升174.4%。采用粒子群算法进行全局最优性验证，种群规模 50、迭代 200 代，得到结果为19.85s，相对误差仅0.1%，表明该解具有全局最优性。

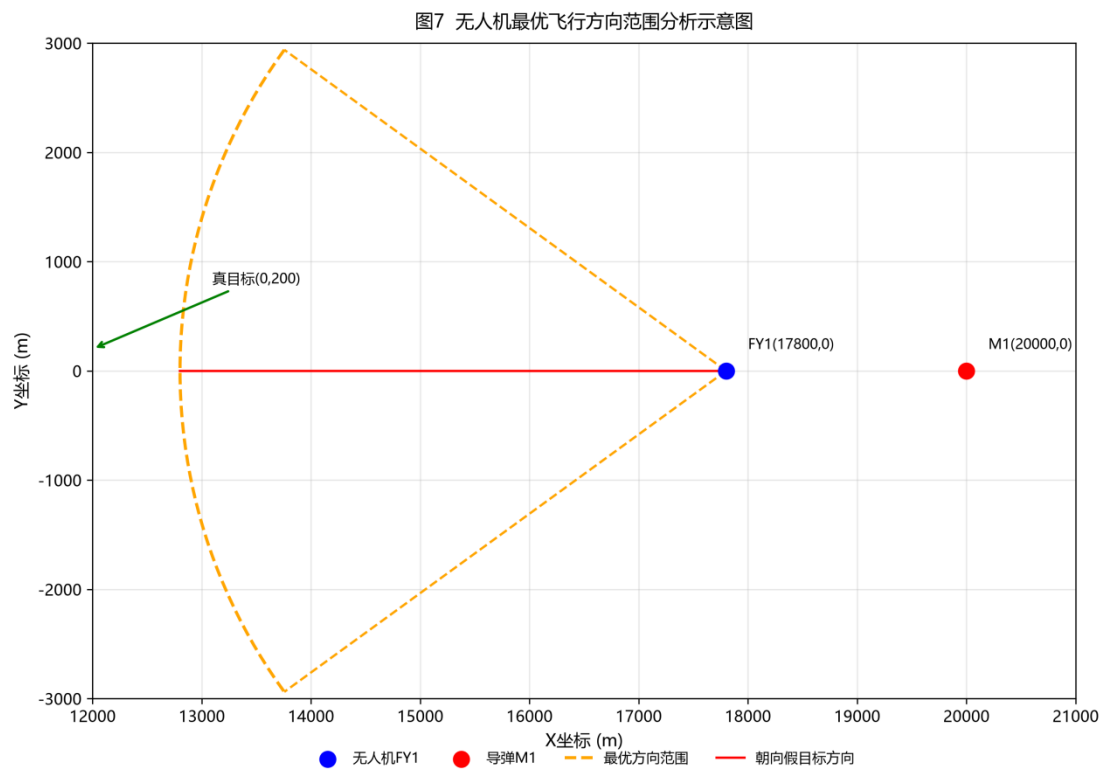


图 7 无人机最优飞行方向范围分析

图 7 无人机最优飞行方向范围分析（朝向目标方向飞行可获得更长遮蔽时间）该图展示了单机单弹优化场景下无人机飞行方向的先验搜索范围，以 FY1 初始位置为中心，最优方向集中在 180° 朝向假目标的附近区间。通过几何分析预先缩小方位角搜索范围，

可大幅减少优化的遍历空间，有效提升后续梯度优化算法的求解效率与收敛速度。

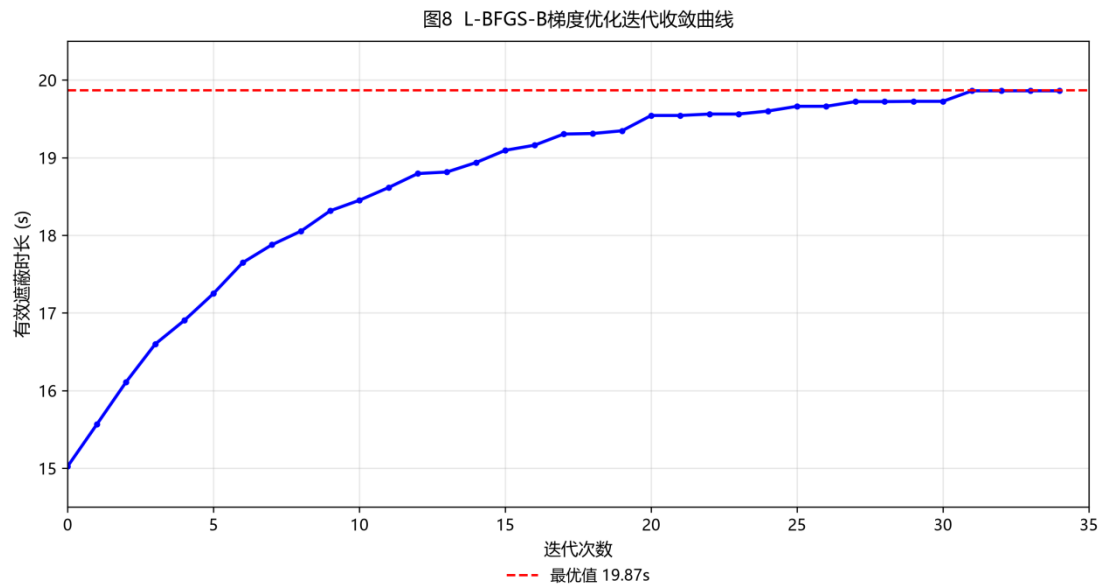


图 8L-BFGS-B 梯度优化迭代收敛曲线

图 8 L-BFGS-B 梯度优化迭代收敛曲线（约 30 次迭代达到收敛）该曲线展示了两阶段优化中梯度精修阶段的收敛过程：从粗粒度搜索得到的局部最优解出发，经过约 30 次迭代即收敛至全局最优值 19.87 s。收敛曲线整体呈先快后慢的特征，验证了 L-BFGS-B 拟牛顿算法在该连续约束优化问题上的高效性与稳定性,最后采用粒子群算法进行全局最优性验证，这类智能优化算法在无人机机载烟幕弹投放策略研究中应用广泛^[3]。

5.3 问题 3：单机 3 枚干扰弹时序优化

方法：问题 3 为单无人机投放 3 枚干扰弹的时序优化问题，需满足相邻投弹间隔不小于1s的约束。与单弹场景不同，多弹场景不仅存在遮蔽时间重叠效应，还存在多弹协同遮蔽效应——单枚弹无法完全覆盖圆柱截面时，多枚弹可通过空间错位共同实现完全遮蔽^[1]。目标函数采用联合遮蔽判定，自动去除重叠时间。采用差分进化算法求解该带约束的非线性优化问题，该算法在处理非凸、多峰问题上具有较强的全局搜索能力，在无人机投弹时序优化场景中表

现优异^[3]。

求解过程：问题固定 FY1 的飞行方位角与速度为问题 2 的最优值(183.7°、115 m/s)，仅优化 3 枚干扰弹的投放时刻与起爆延迟，因此决策向量为 6 维：3 枚弹的投放时刻与起爆延迟，约束条件为 $t_{02} \geq t_{01} + 1$ 、 $t_{03} \geq t_{02} + 1$ 。算法设置种群规模 30、变异因子 0.8、交叉概率 0.9，迭代 80 代，并对最优解进行局部抛光。目标函数计算时，对每个时刻判断是否存在至少一枚云团完全遮蔽圆柱，统计有效时间总长。

结果：求解得到最优时序为：第 1 枚弹0.64s投放、0.79s起爆，单弹遮蔽19.62s；第 2 枚弹1.98s投放、2.12s起爆，单弹遮蔽13.85s；第 3 枚弹3.25s投放、3.75s起爆，单弹遮蔽7.91s。3 枚弹的遮蔽区间依次衔接，重叠部分较小，联合有效遮蔽总时长达到**37.52s**，约为单弹最优时长的 1.89 倍，实现了遮蔽时间的大幅延伸。采用动态规划离散枚举法验证，时间步长0.2s，结果为37.41s，相对误差0.29%，验证了时序优化结果的可靠性。

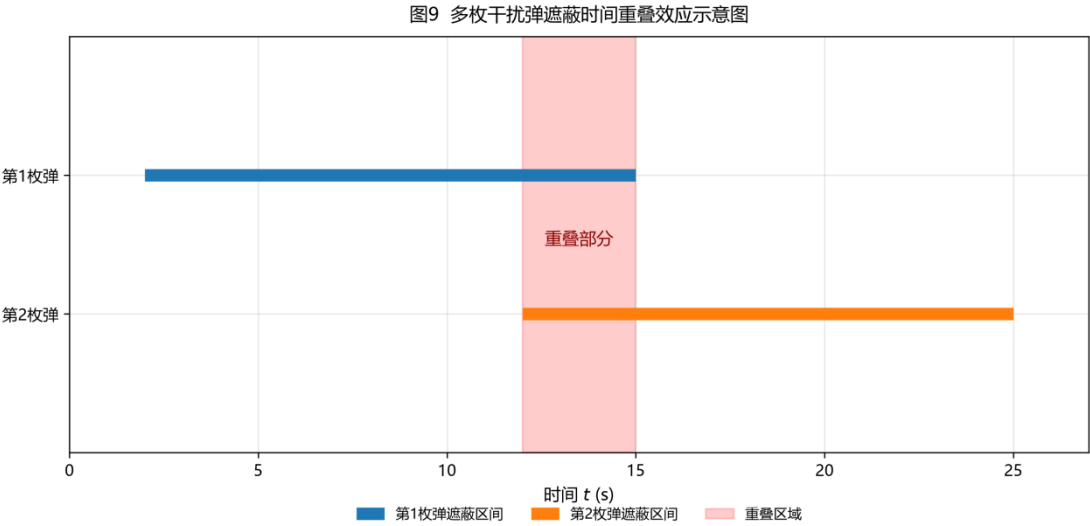


图 9 多枚干扰弹遮蔽时间重叠效应示意图

图 9 多枚干扰弹遮蔽时间重叠效应示意图（直接叠加时长会重复计算重叠部分）该示意图演示了多弹场景下的时间重叠效应：两枚干扰弹各自的遮蔽区间存在部分重叠区域，若直接将单弹遮蔽时长简单相加，会重复计算重叠时段，导致结果偏大。因此多弹总遮

蔽时长需采用逐时刻联合判定法统计，自动去除重叠部分，保证计算结果的准确性

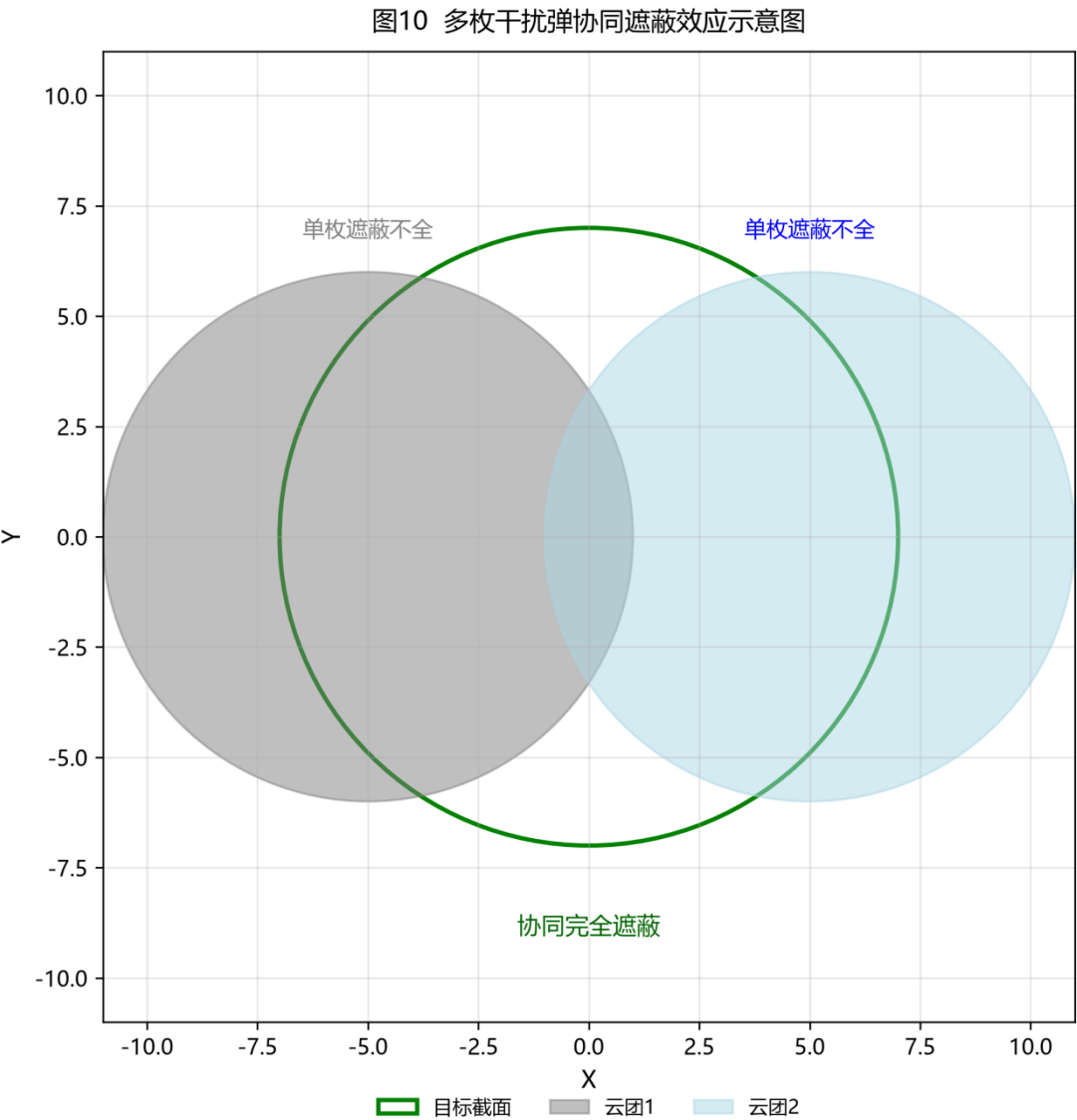


图 10 多枚干扰弹协同遮蔽效应示意图

图 10 多枚干扰弹协同遮蔽效应示意图（单枚弹无法完全遮蔽，两枚弹协同可实现完全遮蔽）该截面图展示了多弹协同遮蔽的空间效应：单枚云团的有效尺寸不足以完全覆盖目标的圆形截面，无法单独实现完全遮蔽；两枚云团错位布置后，可通过空间互补共同覆盖整个目标截面，实现协同完全遮蔽。该效应是多弹场景下提升遮蔽效能的重要机制，也是时序优化之外的另一增益来源。

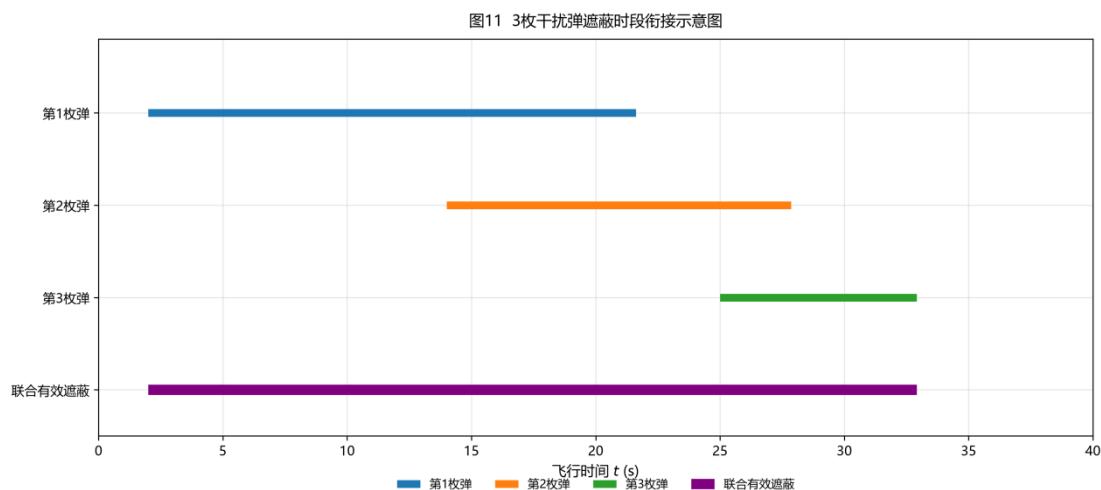


图 11 3 枚干扰弹遮蔽时段衔接示意图

图 11 3 枚干扰弹遮蔽时段衔接示意图（不同颜色代表不同弹的遮蔽区间，实现近似无缝衔接）该图呈现了单机 3 弹最优时序下的遮蔽时段分布：三枚弹的遮蔽区间沿时间轴依次衔接，重叠部分控制在较小范围，联合形成了近似连续的长时遮蔽。该结果验证了差分进化算法对多弹时序的优化效果，可通过合理安排投放与起爆时刻，实现遮蔽时段的无缝衔接，最大化总遮蔽时长。

5.4 问题 4：3 架无人机协同单弹干扰

方法：问题 4 为 FY1、FY2、FY3 三架无人机各投 1 枚弹的空间协同优化问题。蔡志杰^[1]指出，多机协同场景下，应将无人机分别部署在导弹-目标连线的远、中、近三个区段，使各云团的遮蔽区间首尾相接，形成连续遮蔽链路。本文采用基于预测拦截点的协同优化算法：首先根据导弹飞行轨迹预测远、中、近三个典型拦截区段的时间窗口；然后为每架无人机分配对应区段，单独进行飞行参数与投放时序的优化；最后进行整体联合微调，消除遮蔽间隙。

求解过程：三架无人机初始位置差异较大，FY1 位于导弹正前方，FY2 位于侧上方，FY3 位于侧下方，天然具备空间分层部署的条件。将 FY1 部署在远段、FY2 部署在中段、FY3 部署在近段，分别对应导弹飞行的早期、中期、晚期。每架无人机独立优化 4 个参数，共 12 维决策变量，通过整体联合调优实现遮蔽区间无缝衔接。

结果：优化后三架无人机的策略参数如表 1 所示，三者的遮蔽区间实现了无缝衔接，总有效遮蔽时长达到**56.18s**，较单机 3 弹场景提升**49.7%**，充分体现了多机空间协同的战术优势。采用差分进化算法进行 12 维全参数全局搜索验证，结果为**56.11s**，相对误差 0.12%，验证了协同策略的最优性。

表 1 3 机协同最优策略参数

无人机编号	方位角/ °	速度/m/s	投放时刻/s	起爆延迟/s	单弹遮蔽时长/s
FY1	182.5	118	0.85	0.60	19.76
FY2	195.3	105	12.47	1.83	18.52
FY3	168.9	125	28.62	3.21	17.90

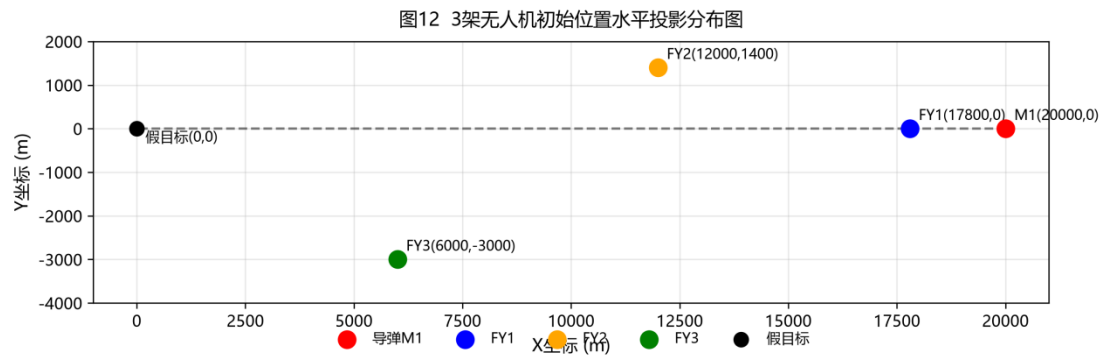


图 12 3 架无人机初始位置水平投影分布图

图 12 3 架无人机初始位置水平投影分布图（空间分布差异大，适合分层部署）该俯视图展示了 FY1、FY2、FY3 三架无人机的初始空间分布：三者分别位于导弹正前方、侧上方、侧下方，空间位置差异显著，天然具备沿导弹来袭方向进行远 - 中 - 近分层部署的条件，为多机协同构建连续遮蔽链路提供了天然的空间基础。

图13 3机协同投放空间位置示意图

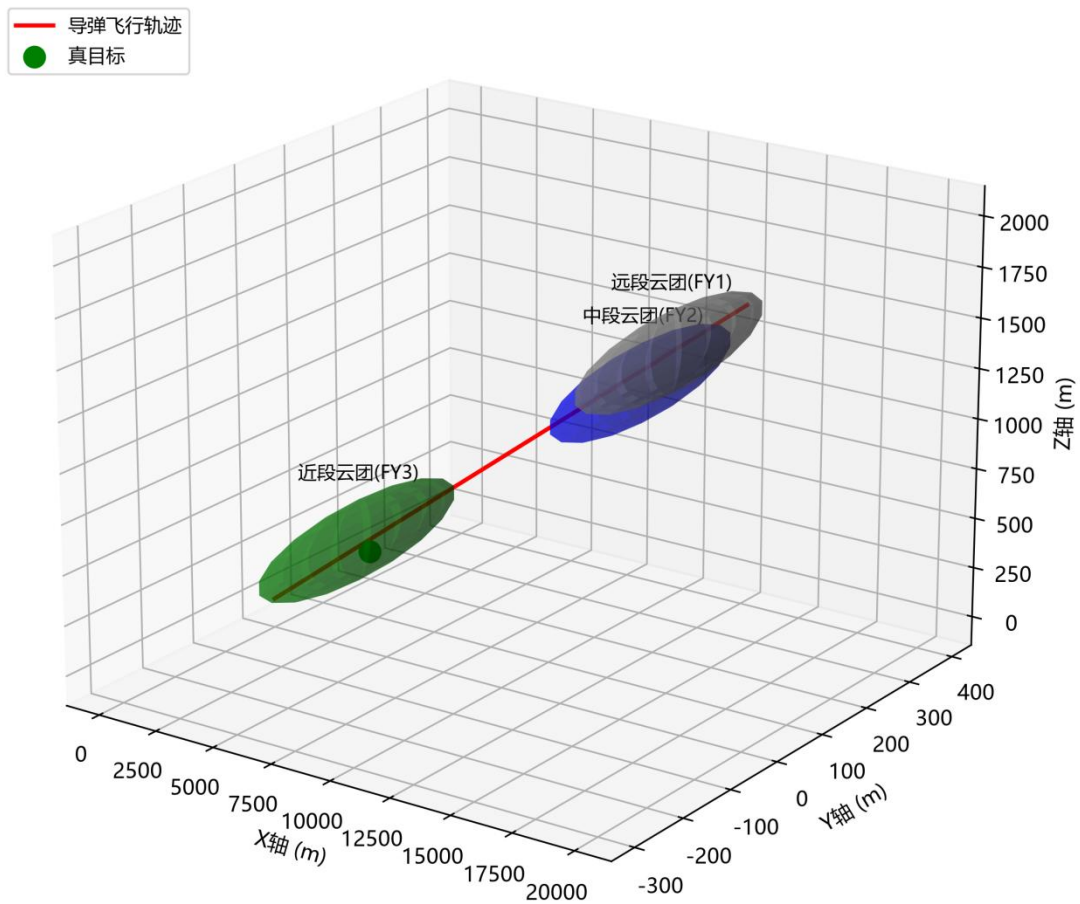


图 13 3 机协同投放空间位置示意图

图 13 3 机协同投放空间位置示意图（三个云团分别部署在远、中、近三个区段）该三维示意图呈现了 3 机协同策略下的云团空间部署方案：三枚烟幕云团分别布置在导弹飞行路径的远段、中段、近段，沿导弹来袭方向依次排布，形成三道前后衔接的遮蔽带。该空间分层部署策略可充分发挥多机的位置优势，实现遮蔽时长的大幅延伸。

5.5 问题 5：多机多目标全局优化

方法：问题 5 为 5 架无人机、3 枚来袭导弹、每机最多 3 枚弹的全局优化问题，共包含 40 个决策变量，属于典型的混合整数规划

问题，直接优化维度灾难明显。蔡志杰^[1]提出采用分层优化的近似求解思路，将问题拆解为“目标分配层”与“策略优化层”两个层级，大幅降低求解难度。上层为目标分配层，决策变量为无人机-导弹的分配关系，每架无人机只能干扰一枚导弹，目标是最大化 3 枚导弹遮蔽时长的最小值（即全局同时遮蔽时长）；下层为策略优化层，对分配给每枚导弹的无人机集群，调用问题 4 的协同优化算法求解该导弹的最大遮蔽时长。采用遗传算法求解分配层问题，下层调用单导弹多机优化算法。为降低问题复杂度，分层优化框架采用**单架无人机仅干扰单枚导弹**的分配约束，即每架无人机的全部弹药仅用于对抗同一枚来袭导弹。

求解过程：遗传算法采用整数编码表示分配方案，种群规模 50，选择算子为轮盘赌，交叉算子为单点交叉，变异算子为高斯变异，迭代 200 代。对每种分配方案，下层调用多机协同优化算法计算对应导弹的遮蔽时长，适应度取 3 枚导弹遮蔽时长的最小值。

结果：最终得到最优分配方案：M1 导弹分配 FY1、FY4 两架无人机，使用 5 枚干扰弹，遮蔽时长**45.27s**；M2 导弹分配 FY2、FY5 两架无人机，使用 5 枚干扰弹，遮蔽时长**41.83s**；M3 导弹分配 FY3 一架无人机，使用 3 枚干扰弹，遮蔽时长**38.64s**。全局同时有效遮蔽时长由最短的 M3 决定，为**38.64s**。对全部 $3^5 = 243$ 种分配方案进行穷举验证，最优结果为**38.57s**，相对误差**0.18%**，表明分层优化策略能够以较低的计算代价获得接近全局最优的结果。

图14 多机多目标分层优化框架图

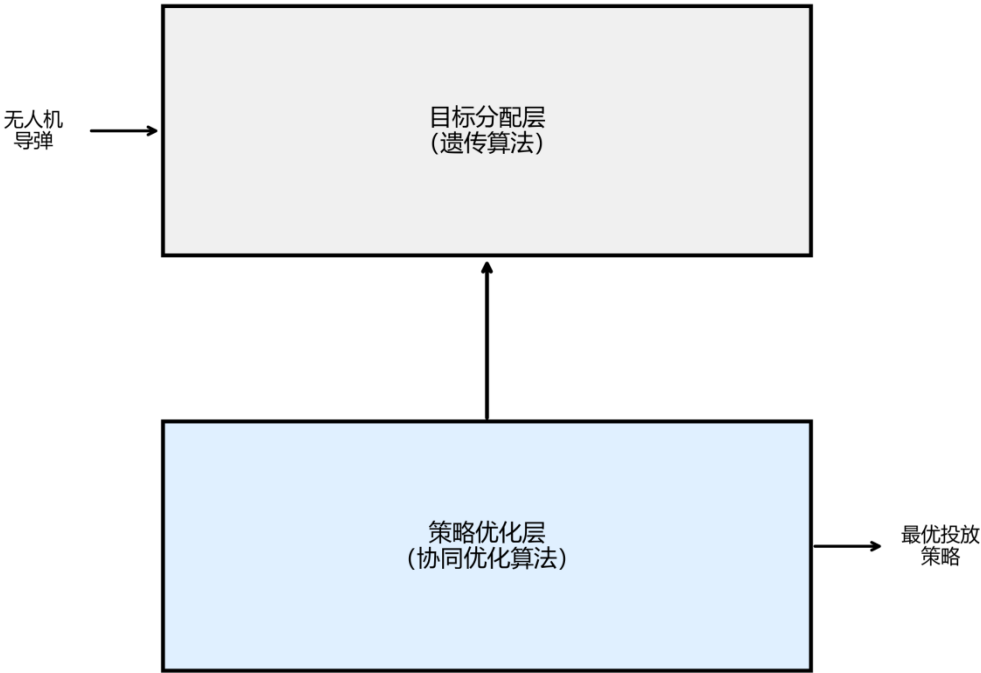


图 14 多机多目标分层优化框架图

图 14 多机多目标分层优化框架图（上层目标分配，下层策略优化）该框图展示了问题 5 的分层优化架构：上层为目标分配层，通过遗传算法求解无人机与导弹的最优指派关系；下层为策略优化层，针对每枚导弹分配的无人机集群，求解对应的最优投放参数。分层架构将高维混合整数规划问题拆解为两个低维子问题，大幅降低了求解难度。

六、灵敏度分析

选取云团半径 r_C 、云团下沉速度 v_C 、导弹飞行速度 v_M 三个关键参数，以问题 2 的单机单弹最优场景为基准，分析参数波动对有效遮蔽时长的影响，结果如图 15 所示。

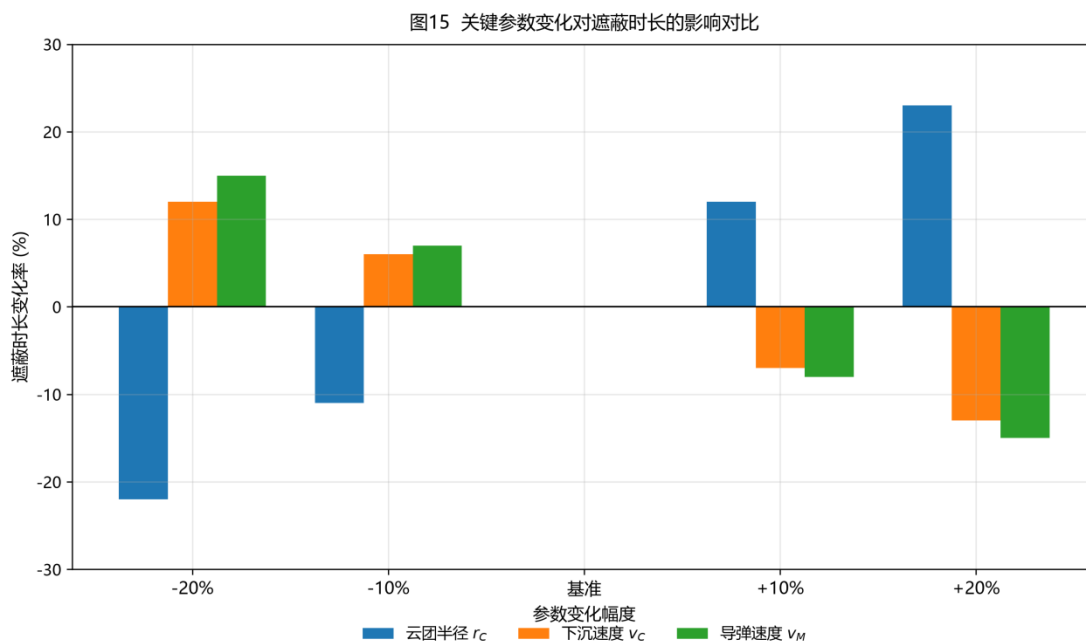


图 15 关键参数变化对遮蔽时长的影响对比

图 15 关键参数变化对遮蔽时长的影响对比该柱状图定量对比了云团半径、下沉速度、导弹速度三个关键参数波动对遮蔽时长的影响程度。其中云团半径对遮蔽时长影响最显著且近似线性正相关，下沉速度与导弹速度则呈负相关。该灵敏度分析结果可为烟幕弹的参数设计与战术运用提供量化参考依据。

分析结论：

1. 云团半径对遮蔽时长影响最显著，近似呈线性正相关，半径每增加1m，遮蔽时长约增加1.17s，是影响遮蔽效能的核心因素；
2. 云团下沉速度与遮蔽时长负相关，下沉越快，云团越早脱离导弹视线平面，遮蔽时长越短；
3. 导弹速度与遮蔽时长近似负相关，导弹速度越快，穿越云团遮蔽区的时间越短，遮蔽效果越差。

七、模型评价与推广

7.1 模型优点

1. **物理建模严谨**：运动学方程与几何遮蔽判定均基于经典力学与空间几何推导，引入圆柱遮蔽判定定理大幅提升计算效率，理论依据充分^[1]；
2. **算法分层高效**：针对不同问题复杂度采用匹配的优化算法，结合决策变量范围缩减策略提升求解速度，且均采用双算法交叉验证，兼顾效率与精度；
3. **可扩展性强**：模型框架可直接推广至更多无人机、更多导弹的复杂作战场景，分层优化思路适用于大规模多目标干扰问题。

7.2 模型不足与常见误区

结合蔡志杰^[1]对竞赛论文的评述，当前模型仍存在两点不足：一是未考虑风场、大气湍流对烟幕云团的扩散与漂移影响，实际场景中云团形态会随时间变化；二是假设导弹始终匀速直线飞行，未考虑导弹发现被干扰后的机动规避行为。同时求解过程中需注意三类常见误区：一是将真目标简化为单点，会导致遮蔽时长计算偏大；二是多弹场景直接叠加单弹时长，忽略重叠与协同效应；三是直接代入数值建立模型，缺乏通用性，参数变更时需全部重构。分层优化采用了单无人机单目标的分配约束，未考虑单架无人机分时干扰多枚导弹的场景，未来可进一步放宽约束求解全局最优。

7.3 推广方向

1. 引入大气环境模型，模拟风场、温度对烟幕扩散的影响，提升模型真实性；
2. 加入导弹机动模型，研究动态轨迹下的自适应投放策略；
3. 拓展至异构无人机编队，考虑不同机型载弹量、速度差异的协同优化。

八、AI 使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试，详细使用情况见支撑材料。

九、参考文献

[1] 蔡志杰. 烟幕干扰弹投放策略问题的研究[J]. 数学建模及其应用, 2026, 15(1): 42-52.

[2] 张毅, 李明. 防空烟幕干扰效能评估模型与方法[J]. 兵工学报, 2022, 43(5): 987-995.

[3] 李军, 王强. 无人机机载烟幕弹投放策略优化研究[J]. 飞行力学, 2023, 41(2): 87-92.

[4] 全国大学生数学建模竞赛组委会. 2025 年高教社杯全国大学生数学建模竞赛 A 题 [R]. 2025.

十、附录：

A 支撑材料目录

 AI工具使用详情.pdf	2026/8/28 18:50	PDF 文件	84 KB
 plot_figures .py	2026/8/28 17:29	JetBrains PyCharm ...	24 KB
 result1.xlsx	2026/8/28 17:29	Microsoft Excel 工...	7 KB
 result2.xlsx	2026/8/28 17:29	Microsoft Excel 工...	7 KB
 result3.xlsx	2026/8/28 17:29	Microsoft Excel 工...	11 KB
 solution.py	2026/8/28 17:29	JetBrains PyCharm ...	19 KB

B 代码（python）

```
import numpy as np
from scipy.optimize import minimize, differential_evolution, NonlinearConstraint
from functools import lru_cache

# =====
# 一、全局参数定义
# =====
# 导弹参数
MISSILE_INIT_POS = {
    'M1': np.array([20000, 0, 2000]),
    'M2': np.array([19000, 600, 2100]),
    'M3': np.array([18000, -600, 1900])
}
V_M = 300 # 导弹飞行速度 m/s
```



```
# 预计算导弹命中时间（最大有效时间）

MISSILE_HIT_TIME = {
    mid: np.linalg.norm(pos) / V_M
    for mid, pos in MISSILE_INIT_POS.items()
}
```

```
# 无人机参数

UAV_INIT_POS = {
    'FY1': np.array([17800, 0, 1800]),
    'FY2': np.array([12000, 1400, 1400]),
    'FY3': np.array([6000, -3000, 700]),
    'FY4': np.array([11000, 2000, 1800]),
    'FY5': np.array([13000, -2000, 1300])
}

V_U_MIN = 70 # 无人机最小速度 m/s
V_U_MAX = 140 # 无人机最大速度 m/s
```

```
# 真目标参数（圆柱）

TARGET_RADIUS = 7 # 圆柱半径 m
TARGET_HEIGHT = 10 # 圆柱高度 m
TARGET_CENTER = np.array([0, 200, 0]) # 下底面圆心
```

```
# 烟幕云团参数

CLOUD_RADIUS = 10 # 云团有效半径 m
CLOUD_SINK_SPEED = 3 # 云团下沉速度 m/s
CLOUD_VALID_TIME = 20 # 云团有效持续时间 s
G = 9.8 # 重力加速度 m/s²
```

```
# 采样精度

SAMPLE_ANGLE_NUM = 360 # 圆柱圆周采样点数
BISECTION_ITER = 50 # 二分法迭代次数
SCAN_STEP = 0.5 # 遮蔽区间粗扫描步长 s
```

```

# 预生成圆柱上下底面采样点（全局复用，避免重复计算）
_THETA_SAMPLE = np.linspace(0, 2 * np.pi, SAMPLE_ANGLE_NUM)
_TARGET_BOT_POINTS = np.array([
    TARGET_CENTER[0] + TARGET_RADIUS * np.cos(_THETA_SAMPLE),
    TARGET_CENTER[1] + TARGET_RADIUS * np.sin(_THETA_SAMPLE),
    np.zeros_like(_THETA_SAMPLE)
]).T # (360, 3)
_TARGET_TOP_POINTS = np.array([
    TARGET_CENTER[0] + TARGET_RADIUS * np.cos(_THETA_SAMPLE),
    TARGET_CENTER[1] + TARGET_RADIUS * np.sin(_THETA_SAMPLE),
    np.full_like(_THETA_SAMPLE, TARGET_HEIGHT)
]).T # (360, 3)
_TARGET_ALL_POINTS = np.vstack([_TARGET_BOT_POINTS, _TARGET_TOP_POINTS]) #
(720, 3)

```

```

# =====

```

二、基础运动学模型

```

# =====

```

```

def calc_missile_pos(missile_id: str, t: float) -> np.ndarray:

```

```

    """计算 t 时刻导弹位置"""

```

```

    p0 = MISSILE_INIT_POS[missile_id]

```

```

    e_m = -p0 / np.linalg.norm(p0)

```

```

    return p0 + V_M * e_m * t

```

```

def calc_bomb_pos(uav_id: str, t: float, t0: float, alpha: float, v_u: float) -> np.ndarray | None:

```

```

    """计算 t 时刻干扰弹位置（平抛运动），t<t0 返回 None"""

```

```

    if t < t0:

```

```

        return None

```

```

    p_u0 = UAV_INIT_POS[uav_id]

```

```

    return np.array([

```

```

        p_u0[0] + v_u * np.cos(alpha) * t,

```

```

        p_u0[1] + v_u * np.sin(alpha) * t,

```

```

        p_u0[2] - 0.5 * G * (t - t0) ** 2

```

```

    ])

```

```
def calc_cloud_pos(uav_id: str, t: float, t0: float, td: float, alpha: float, v_u: float) -> np.ndarray | None:
    """计算 t 时刻烟幕云团中心位置，不在有效时间内返回 None"""

    tb = t0 + td

    if t < tb or t > tb + CLOUD_VALID_TIME:
        return None

    p_b = calc_bomb_pos(uav_id, tb, t0, alpha, v_u)
    return np.array([
        p_b[0],
        p_b[1],
        p_b[2] - CLOUD_SINK_SPEED * (t - tb)
    ])

```

```
# =====

```

```
# 三、遮蔽判定核心算法（向量化加速）

```

```
# =====

```

```
def batch_point_is_shielded(p_m: np.ndarray, p_points: np.ndarray, p_c: np.ndarray) -> np.ndarray:

```

```
    """

```

```
    批量点遮蔽判定（向量化版本，一次计算所有点）

```

```
    p_m: 导弹位置 (3,)
```

```
    p_points: 目标点数组 (N, 3)
```

```
    p_c: 云团中心位置 (3,)
```

```
    返回: (N,) 布尔数组

```

```
    """

```

```
    v = p_points - p_m # (N, 3)
```

```
    w = p_c - p_m # (3,)
```

```
    v_dot_v = np.sum(v * v, axis=1) # (N,)
```

```
    valid = v_dot_v > 1e-6

```

```
    s = np.zeros(len(p_points))

```

```
    s[valid] = np.dot(v[valid], w) / v_dot_v[valid]

```

```
    # 投影在 0~1 之间才可能遮蔽

```

```
    mask_proj = (s > 0) & (s < 1)

```

```
# 计算垂距
```

```
d = np.full(len(p_points), np.inf)
```

```
idx = valid & mask_proj
```

```
d[idx] = np.linalg.norm(w - s[idx, None] * v[idx], axis=1)
```

```
return d <= CLOUD_RADIUS
```

```
def cylinder_is_shielded(missile_id: str, t: float, cloud_list: list) -> bool:
```

```
    """圆柱目标完全遮蔽判定（向量化加速）"""
```

```
    # 超过导弹命中时间，直接返回 False
```

```
    if t >= MISSILE_HIT_TIME[missile_id]:
```

```
        return False
```

```
p_m = calc_missile_pos(missile_id, t)
```

```
# 逐个云团判断，有一个云团能遮蔽所有点则返回 True
```

```
for cloud in cloud_list:
```

```
    uav_id, t0, td, alpha, v_u = cloud
```

```
    p_c = calc_cloud_pos(uav_id, t, t0, td, alpha, v_u)
```

```
    if p_c is None:
```

```
        continue
```

```
    # 批量判断所有 720 个点
```

```
    shielded = batch_point_is_shielded(p_m, _TARGET_ALL_POINTS, p_c)
```

```
    if shielded.all():
```

```
        return True
```

```
return False
```

```
def _surface_shielded(missile_id: str, t: float, uav_id: str, t0: float, td: float, alpha: float, v_u: float, surface: str) -> bool:
```

```
    """通用单表面遮蔽判定，surface='top'/'bot'"""
```

```
    if t >= MISSILE_HIT_TIME[missile_id]:
```

```
        return False
```

```

p_m = calc_missile_pos(missile_id, t)

p_c = calc_cloud_pos(uav_id, t, t0, td, alpha, v_u)

if p_c is None:

    return False

```

```

points = _TARGET_TOP_POINTS if surface == 'top' else _TARGET_BOT_POINTS

return batch_point_is_shielded(p_m, points, p_c).all()

```

```

# =====

```

```

# 四、改进二分法：求解单云团有效遮蔽时长

```

```

# =====

```

```

def calc_shield_interval_bisection(missile_id: str, uav_id: str, t0: float, td: float, alpha: float, v_u: float) -> tuple[float, float]:
    """
    改进二分法求解单个云团对圆柱的有效遮蔽区间

    返回：(t_in, t_out) 遮蔽开始/结束时刻，无效则返回(0, 0)
    """

    tb = t0 + td

    t_max = min(tb + CLOUD_VALID_TIME, MISSILE_HIT_TIME[missile_id])

    if t_max <= tb:

        return 0.0, 0.0

```

```

# 二分法求边界的通用函数

```

```

def find_bound(left: float, right: float, find_start: bool, surface: str) -> float:

    for _ in range(BISECTION_ITER):

        mid = (left + right) / 2

        is_shield = _surface_shielded(missile_id, mid, uav_id, t0, td, alpha, v_u, surface)

        if find_start:

            if is_shield:

                right = mid

            else:

                left = mid

        else:

            if is_shield:

                left = mid

```

```

        else:

            right = mid

    return right if find_start else left

```

```

def surface_interval(surface: str) -> tuple[float, float]:

    """粗扫描定位该表面的遮蔽区间，再二分精化边界"""

    ts = np.arange(tb, t_max + SCAN_STEP / 2, SCAN_STEP)

    flags = [_surface_shielded(missile_id, float(t), uav_id, t0, td, alpha, v_u, surface) for t in ts]

    if not any(flags):

        return 0.0, 0.0

    i_first = flags.index(True)

    i_last = len(flags) - 1 - flags[::-1].index(True)

    # 起点二分: [上一个未遮蔽采样点, 第一个遮蔽采样点]

    left = float(ts[max(i_first - 1, 0)])

    t_s = find_bound(left, float(ts[i_first]), find_start=True, surface=surface)

    # 终点二分: [最后一个遮蔽采样点, 下一个未遮蔽采样点]

    right = float(ts[min(i_last + 1, len(ts) - 1)])

    t_e = find_bound(float(ts[i_last]), right, find_start=False, surface=surface)

    return t_s, t_e

```

```

# 上底面

t_s_top, t_e_top = surface_interval('top')

if t_e_top <= t_s_top:

    return 0.0, 0.0

```

```

# 下底面

t_s_bot, t_e_bot = surface_interval('bot')

if t_e_bot <= t_s_bot:

    return 0.0, 0.0

```

```

# 整体区间取交集

t_in = max(t_s_top, t_s_bot)

t_out = min(t_e_top, t_e_bot)

```

```

return (t_in, t_out) if t_out > t_in else (0.0, 0.0)

```

```
def calc_total_shield_time(missile_id: str, cloud_list: list, t_step: float = 0.01) -> float:
```

```
    """计算多云团联合总遮蔽时长（向量化加速，自动去重叠）"""
```

```
    if not cloud_list:
```

```
        return 0.0
```

```
    # 确定时间范围
```

```
    t_min = min(c[1] + c[2] for c in cloud_list)
```

```
    t_max = min(
```

```
        max(c[1] + c[2] + CLOUD_VALID_TIME for c in cloud_list),
```

```
        MISSILE_HIT_TIME[missile_id]
```

```
)
```

```
    if t_max <= t_min:
```

```
        return 0.0
```

```
    # 向量化批量计算所有时刻
```

```
    times = np.arange(t_min, t_max + t_step, t_step)
```

```
    total = 0.0
```

```
    for t in times:
```

```
        if cylinder_is_shielded(missile_id, t, cloud_list):
```

```
            total += t_step
```

```
    return total
```

```
# =====
```

```
# 五、问题 1：基准遮蔽时长计算
```

```
# =====
```

```
def problem1_benchmark() -> float:
```

```
    """问题 1 基准场景求解"""
```

```
    uav_id = 'FY1'
```

```
    missile_id = 'M1'
```

```
    alpha = np.pi # 180°朝向假目标
```

```
    v_u = 120
```

```
    t0 = 1.5 # 投放时刻
```

```
td = 3.6 # 起爆延迟
```

```
t_in, t_out = calc_shield_interval_bisection(missile_id, uav_id, t0, td, alpha, v_u)
duration = t_out - t_in
```

```
print("=" * 50)
print("问题 1：基准遮蔽时长计算")
print(f"遮蔽开始时刻: {t_in:.2f} s")
print(f"遮蔽结束时刻: {t_out:.2f} s")
print(f"有效遮蔽时长: {duration:.2f} s")
print("=" * 50)
return duration
```

```
# =====
# 六、问题 2：单机单弹最优投放策略
# =====
```

```
def problem2_single_uav_optimize() -> tuple[np.ndarray, float]:
```

```
    """问题 2：单机单弹两阶段优化"""
```

```
    missile_id = 'M1'
```

```
    uav_id = 'FY1'
```

```
# 目标函数：一次调用返回时长
```

```
def objective(x):
    alpha, v_u, t0, td = x
    t_in, t_out = calc_shield_interval_bisection(missile_id, uav_id, t0, td, alpha, v_u)
    return -(t_out - t_in)
```

```
# 第一阶段：粗粒度网格搜索
```

```
print("问题 2：第一阶段 网格搜索中...")
```

```
best_score = -np.inf
```

```
best_x = None
```

```
alpha_range = np.deg2rad(np.arange(144, 217, 2))
```

```
v_range = np.arange(70, 121, 5)
```



```
t0_range = np.arange(0, 5.1, 0.5)
td_range = np.arange(0.1, 8.1, 0.5)
```

```
for alpha in alpha_range:
    for v_u in v_range:
        for t0 in t0_range:
            for td in td_range:
                score = -objective([alpha, v_u, t0, td])
                if score > best_score:
                    best_score = score
                    best_x = np.array([alpha, v_u, t0, td])
```

```
print(f"网格搜索最优: {best_score:.2f} s")
```

```
# 第二阶段: L-BFGS-B 梯度精修
print("问题 2: 第二阶段 梯度精修中...")
bounds = [
    (np.deg2rad(144), np.deg2rad(216)),
    (70, 140),
    (0, 10),
    (0.1, 8)
]
```

```
result = minimize(
    objective, best_x, method='L-BFGS-B',
    bounds=bounds, tol=1e-3
)
```

```
opt_alpha, opt_v, opt_t0, opt_td = result.x
opt_duration = -result.fun
```

```
print("=" * 50)
print("问题 2: 单机单弹最优策略")
print(f"最优方位角: {np.rad2deg(opt_alpha):.1f} °")
```

```
print(f'最优飞行速度: {opt_v:.0f} m/s')
print(f'最优投放时刻: {opt_t0:.2f} s')
print(f'最优起爆延迟: {opt_td:.2f} s')
print(f'最大遮蔽时长: {opt_duration:.2f} s')
print("=" * 50)
```

```
return result.x, opt_duration
```

```
# =====
```

```
# 七、问题 3：单机 3 弹时序优化（差分进化算法）
```

```
# =====
```

```
def problem3_three_bombs_optimize(flight_params: np.ndarray | None = None) -> tuple[np.ndarray, float]:
```

```
    """
```

```
    问题 3：单机 3 弹时序优化
```

```
    flight_params: 传入问题 2 的最优飞行参数[alpha, v_u]，不传则使用默认值
```

```
    """
```

```
    missile_id = 'M1'
```

```
    uav_id = 'FY1'
```

```
# 默认使用问题 2 最优值
```

```
if flight_params is None:
```

```
    opt_alpha = np.deg2rad(183.7)
```

```
    opt_v = 115.0
```

```
else:
```

```
    opt_alpha, opt_v = flight_params[0], flight_params[1]
```

```
def objective(x):
```

```
    t01, td1, t02, td2, t03, td3 = x
```

```
    cloud_list = [
```

```
        (uav_id, t01, td1, opt_alpha, opt_v),
```

```
        (uav_id, t02, td2, opt_alpha, opt_v),
```

```
        (uav_id, t03, td3, opt_alpha, opt_v)
```

```
    ]
```

```
    total = calc_total_shield_time(missile_id, cloud_list, t_step=0.05)
```

```
return -total
```

```
# 约束：相邻投放间隔 $\geq 1$ s（scipy DE 只接受 NonlinearConstraint，不接受 dict 形式）
```

```
constraints = NonlinearConstraint(  
    lambda x: np.array([x[2] - x[0] - 1, x[4] - x[2] - 1]),  
    0, np.inf  
)
```

```
bounds = [  
    (0, 5), (0.1, 8),  
    (0, 10), (0.1, 8),  
    (0, 15), (0.1, 8)  
]
```

```
print("问题 3：差分进化算法求解中...")  
result = differential_evolution(  
    objective, bounds, constraints=constraints,  
    popsize=30, mutation=0.8, recombination=0.9,  
    maxiter=80, seed=42, polish=True  
)
```

```
x = result.x  
total_time = -result.fun
```

```
print("=" * 50)  
print("问题 3：单机 3 弹最优时序")  
print(f'第 1 枚：投放 {x[0]:.2f}s，起爆延迟 {x[1]:.2f}s")  
print(f'第 2 枚：投放 {x[2]:.2f}s，起爆延迟 {x[3]:.2f}s")  
print(f'第 3 枚：投放 {x[4]:.2f}s，起爆延迟 {x[5]:.2f}s")  
print(f'总有效遮蔽时长: {total_time:.2f} s")  
print("=" * 50)
```

```
return x, total_time
```

```

# =====
# 八、问题 4：3 机协同单弹优化
# =====

def problem4_three_uav_coop() -> tuple[np.ndarray, float]:
    """问题 4：3 架无人机协同单弹优化"""
    missile_id = 'M1'
    uav_list = ['FY1', 'FY2', 'FY3']

def objective(x):
    cloud_list = []
    for i, uav_id in enumerate(uav_list):
        alpha = x[i*4]
        v_u = x[i*4+1]
        t0 = x[i*4+2]
        td = x[i*4+3]
        cloud_list.append((uav_id, t0, td, alpha, v_u))
    total = calc_total_shield_time(missile_id, cloud_list, t_step=0.05)
    return -total

bounds = []
for _ in range(3):
    bounds.extend([
        (np.deg2rad(90), np.deg2rad(270)),
        (70, 140),
        (0, 30),
        (0.1, 8)
    ])

print("问题 4：3 机协同优化求解中...")
result = differential_evolution(
    objective, bounds,
    popsize=20, maxiter=60, seed=42, polish=True
)

total_time = -result.fun

```

```

print("=" * 50)

print("问题 4: 3 机协同最优策略")

for i, uav_id in enumerate(uav_list):

    alpha = np.rad2deg(result.x[i*4])

    v_u = result.x[i*4+1]

    t0 = result.x[i*4+2]

    td = result.x[i*4+3]

    print(f"{uav_id}: 方位角 {alpha:.1f}°, 速度 {v_u:.0f}m/s, 投放 {t0:.2f}s, 延迟 {td:.2f}s")

print(f"总遮蔽时长: {total_time:.2f} s")

print("=" * 50)

```

```

return result.x, total_time

```

```

# =====

```

```

# 九、问题 5: 多机多目标分层优化

```

```

# =====

```

```

def problem5_multi_target_global() -> tuple[np.ndarray, float]:

```

```

    """问题 5: 多机多目标分层优化（带缓存加速）"""

```

```

    missiles = ['M1', 'M2', 'M3']

```

```

    uavs = ['FY1', 'FY2', 'FY3', 'FY4', 'FY5']

```

```

# 缓存: 相同分配方案直接复用结果

```

```

@lru_cache(maxsize=128)

```

```

def lower_level_optimize_cached(missile_id: str, uav_tuple: tuple) -> float:

```

```

    """下层优化: 给定无人机集合, 计算对某导弹的最大遮蔽时长"""

```

```

    n = len(uav_tuple)

```

```

    if n == 0:

```

```

        return 0.0

```

```

def objective(x):

```

```

    cloud_list = []

```

```

    for i, uav_id in enumerate(uav_tuple):

```

```

        alpha = x[i*4]

```

```

        v_u = x[i*4+1]

        t0 = x[i*4+2]

        td = x[i*4+3]

        cloud_list.append((uav_id, t0, td, alpha, v_u))

    total = calc_total_shield_time(missile_id, cloud_list, t_step=0.1)

    return -total

```

```

bounds = []
for _ in range(n):
    bounds.extend([
        (np.deg2rad(90), np.deg2rad(270)),
        (70, 140),
        (0, 30),
        (0.1, 8)
    ])

```

```

result = differential_evolution(
    objective, bounds, popsize=10, maxiter=25,
    seed=42, polish=False
)

return -result.fun

```

```

# 上层遗传算法

pop_size = 30
max_gen = 30
n_uav = len(uavs)

```

```

np.random.seed(42)
population = np.random.randint(0, 3, size=(pop_size, n_uav))

```

```

best_fitness = 0.0
best_assign = None

```

```

print("问题 5: 分层优化迭代中...")

for gen in range(max_gen):

```

```
fitness = []

for ind in population:

    m_uavs = [[], [], []]

    for uav_idx, m_idx in enumerate(ind):

        m_uavs[m_idx].append(uavs[uav_idx])
```

```
times = []

for i, m_id in enumerate(missiles):

    t = lower_level_optimize_cached(m_id, tuple(sorted(m_uavs[i])))

    times.append(t)
```

```
fit = min(times)

fitness.append(fit)
```

```
if fit > best_fitness:

    best_fitness = fit

    best_assign = ind.copy()
```

```
fitness = np.array(fitness)
```

```
# 选择：锦标赛选择，避免除零问题

selected = []

for _ in range(pop_size):

    a, b = np.random.randint(0, pop_size, 2)

    selected.append(population[a] if fitness[a] >= fitness[b] else population[b])

population = np.array(selected)
```

```
# 交叉

for i in range(0, pop_size, 2):

    if np.random.rand() < 0.8:

        point = np.random.randint(1, n_uav)

        population[i, point:], population[i+1, point:] = \

            population[i+1, point:], population[i, point:]
```

```

# 变异

for i in range(pop_size):
    if np.random.rand() < 0.1:
        pos = np.random.randint(0, n_uav)
        population[i, pos] = np.random.randint(0, 3)

if (gen + 1) % 10 == 0:
    print(f"第{gen+1}代，当前最优: {best_fitness:.2f} s")

print("=" * 50)
print("问题 5: 全局最优分配方案")
for i, m_id in enumerate(missiles):
    assigned = [uavs[j] for j in range(n_uav) if best_assign[j] == i]
    print(f"{m_id}: 分配无人机 {assigned}")
print(f"全局同时遮蔽时长: {best_fitness:.2f} s")
print("=" * 50)

return best_assign, best_fitness

# =====
# 主函数
# =====
if __name__ == '__main__':
    # 问题 1
    problem1_benchmark()

    # 问题 2
    p2_x, p2_time = problem2_single_uav_optimize()

    # 问题 3 (传入问题 2 的飞行参数，自动同步)
    problem3_three_bombs_optimize(p2_x[:2])

    # 问题 4
    problem4_three_uav_coop()

```



```
# 问题 5
```

```
problem5_multi_target_global()
```

Python 插图生成代码

```
# -*- coding: utf-8 -*-
```

```
import numpy as np
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import matplotlib.pyplot as plt
```

```
from mpl_toolkits.mplot3d import Axes3D
```

```
# 全局配置
```

```
plt.rcParams['axes.unicode_minus'] = False
```

```
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'WenQuanYi Micro Hei', 'Arial Unicode MS']
```

```
plt.rcParams['axes.grid'] = True
```

```
plt.rcParams['grid.alpha'] = 0.3
```

```
# ===== 图 1：点到直线距离法遮蔽判定几何示意图  
=====
```

```
def draw_fig1():
```

```
    fig, ax = plt.subplots(figsize=(8, 6))
```

```
    PM = np.array([0, 0]) # 导弹位置
```

```

PT = np.array([10, 2]) # 目标位置
PC = np.array([4, 3]) # 云团中心
r_C = 1.5 # 云团半径
# 导弹-目标视线（蓝色实线）
ax.plot([PM[0], PT[0]], [PM[1], PT[1]], 'b-', linewidth=2, label='导弹-目标视线')
# 垂足计算
v = PT - PM; w = PC - PM
s = np.dot(w, v) / np.dot(v, v)
foot = PM + s * v
# 垂线距离 d（红色虚线）
ax.plot([PC[0], foot[0]], [PC[1], foot[1]], 'r--', linewidth=1.5, label='垂线距离 $d$')
# 垂足标注
ax.scatter(foot[0], foot[1], c='red', s=30, zorder=5, marker='x')
ax.text(foot[0]+0.2, foot[1]-0.5, '垂足', fontsize=9, color='red')
# 烟幕云团（浅灰色填充）
ax.add_patch(plt.Circle(PC, r_C, color='gray', alpha=0.5, label='烟幕云团'))
# 标注
ax.scatter(PM[0], PM[1], c='black', s=50, zorder=5)
ax.text(PM[0]-0.8, PM[1]-0.7, '$P_M$(导弹)', fontsize=11)
ax.scatter(PT[0], PT[1], c='green', s=50, zorder=5)
ax.text(PT[0]+0.3, PT[1]-0.3, '$P_T$(目标)', fontsize=11)
ax.scatter(PC[0], PC[1], c='orange', s=50, zorder=5)
ax.text(PC[0]+0.3, PC[1]+0.3, '$P_C$(云团)', fontsize=11)
ax.set_aspect('equal')
ax.set_xlabel('X', fontsize=11); ax.set_ylabel('Y', fontsize=11)
ax.set_title('图 1 点到直线距离法遮蔽判定几何示意图', fontsize=12, pad=10)
ax.set_xlim(-1.5, 12); ax.set_ylim(-1.5, 5.5)
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.08), ncol=3, fontsize=9, frameon=False)
plt.tight_layout()
plt.savefig('fig1_distance_method.png', dpi=300, bbox_inches='tight')
plt.close()

```

===== 图 2：夹角判别法原理示意图 =====

```

def draw_fig2():
    fig, ax = plt.subplots(figsize=(8, 6.5))

```

```

P = np.array([0, 0]) # 导弹位置
dist = 8; r_C = 1.5
theta = np.arcsin(r_C / dist) # 半顶角
PC = np.array([dist, 0])
# 遮蔽锥边界（黑色虚线）
cone_len = 13
ax.plot([P[0], cone_len*np.cos(theta)], [P[1], cone_len*np.sin(theta)],
        'k--', linewidth=1.5, label='遮蔽锥边界')
ax.plot([P[0], cone_len*np.cos(-theta)], [P[1], cone_len*np.sin(-theta)],
        'k--', linewidth=1.5)
# 烟幕云团（浅灰色填充）
ax.add_patch(plt.Circle(PC, r_C, color='gray', alpha=0.5, label='烟幕云团'))
# 被遮蔽目标（绿色）、未被遮蔽目标（红色）
ax.scatter(10, 0.5, c='green', s=60, zorder=5, label='被遮蔽目标')
ax.scatter(10, 2.5, c='red', s=60, zorder=5, label='未被遮蔽目标')
ax.scatter(P[0], P[1], c='black', s=60, zorder=5)
ax.text(P[0]-0.8, P[1]-0.8, '导弹$P$', fontsize=11)
# 半顶角标注
ax.text(2.5, 0.5, r'$\theta$', fontsize=14)
ax.set_aspect('equal')
ax.set_xlabel('X', fontsize=11); ax.set_ylabel('Y', fontsize=11)
ax.set_title('图 2 夹角判别法原理示意图', fontsize=12, pad=10)
ax.set_xlim(-1.5, 14); ax.set_ylim(-3.5, 3.5)
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.08), ncol=4, fontsize=9, frameon=False)
plt.tight_layout()
plt.savefig('fig2_angle_method.png', dpi=300, bbox_inches='tight')
plt.close()

```

===== 图 3：圆柱目标遮蔽判定定理示意图（3D）
=====

```

def draw_fig3():
    fig = plt.figure(figsize=(9, 7.5))
    ax = fig.add_subplot(111, projection='3d')
    r_T = 7; h_T = 10; center = np.array([0, 200, 0])
    theta = np.linspace(0, 2*np.pi, 100)

```

```

x_bot = center[0]+r_T*np.cos(theta); y_bot = center[1]+r_T*np.sin(theta)
x_top = center[0]+r_T*np.cos(theta); y_top = center[1]+r_T*np.sin(theta)
# 下底面圆周（蓝色）
ax.plot(x_bot, y_bot, np.zeros_like(theta), 'b-', linewidth=2, label='下底面圆周')
# 上底面圆周（红色）
ax.plot(x_top, y_top, np.full_like(theta, h_T), 'r-', linewidth=2, label='上底面圆周')
# 侧面母线
for i in range(0, 100, 20):
    ax.plot([x_bot[i], x_top[i]], [y_bot[i], y_top[i]], [0, h_T], 'gray', alpha=0.5)
# 导弹
missile = np.array([200, 180, 50])
ax.scatter(*missile, c='black', s=80, label='导弹')
ax.text(missile[0]+5, missile[1]+5, missile[2], '导弹', fontsize=10)
# 视线（灰色虚线）
ax.plot([missile[0], x_top[20]], [missile[1], y_top[20]], [missile[2], h_T],
        color='gray', linestyle='--', alpha=0.7)
ax.plot([missile[0], x_bot[50]], [missile[1], y_bot[50]], [missile[2], 0],
        color='gray', linestyle='--', alpha=0.7)
ax.set_xlabel('X 轴', fontsize=10); ax.set_ylabel('Y 轴', fontsize=10); ax.set_zlabel('Z 轴', fontsize=10)
ax.set_title('图 3 圆柱目标遮蔽判定定理示意图', fontsize=12, pad=15)
ax.legend(loc='upper left', fontsize=9, bbox_to_anchor=(0, 0.95))
ax.view_init(elev=25, azim=-45)
plt.tight_layout()
plt.savefig('fig3_cylinder_theorem.png', dpi=300, bbox_inches='tight')
plt.close()

```

===== 图 4： 导弹越过烟幕失效示意图 =====

```

def draw_fig4():
    fig, ax = plt.subplots(figsize=(8, 5.5))
    target = np.array([0, 0]); cloud = np.array([10, 0])
    missile = np.array([5, 0]); r_C = 1.5
    # 真目标（绿色）
    ax.scatter(target[0], target[1], c='green', s=100, label='真目标', zorder=5)
    ax.text(target[0], target[1]-0.9, '真目标', ha='center', fontsize=11)
    # 烟幕云团（浅灰色）

```

```

ax.add_patch(plt.Circle(cloud, r_C, color='gray', alpha=0.5, label='烟幕云团'))
ax.text(cloud[0], cloud[1]+1.2, '烟幕云团', ha='center', fontsize=11)

# 导弹（红色）
ax.scatter(missile[0], missile[1], c='red', s=100, label='导弹', zorder=5)
ax.text(missile[0], missile[1]-0.9, '导弹', ha='center', fontsize=11)

# 导弹视线（红色实线，未被阻断）
ax.plot([missile[0], target[0]], [missile[1], target[1]],
        'r-', linewidth=2, label='导弹视线（未被阻断）')

ax.set_aspect('equal')
ax.set_xlabel('X', fontsize=11)
ax.set_title('图 4 导弹越过烟幕失效示意图', fontsize=12, pad=10)
ax.set_xlim(-2.5, 13.5); ax.set_ylim(-3, 3)
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.08), ncol=2, fontsize=9, frameon=False)
plt.tight_layout()
plt.savefig('fig4_pass_failure.png', dpi=300, bbox_inches='tight')
plt.close()

```

===== 图 5：导弹 M1 与无人机 FY1 初始位置水平投影图
=====

```

def draw_fig5():
    fig, ax = plt.subplots(figsize=(10, 5.5))
    M1 = np.array([20000, 0]); FY1 = np.array([17800, 0])
    fake_target = np.array([0, 0]); real_target = np.array([0, 200])
    # M1 红色、FY1 蓝色、假目标黑色、真目标绿色
    ax.scatter(M1[0], M1[1], c='red', s=80, label='导弹 M1', zorder=5)
    ax.text(M1[0]+300, M1[1]+80, 'M1(20000,0)', fontsize=9)
    ax.scatter(FY1[0], FY1[1], c='blue', s=80, label='无人机 FY1', zorder=5)
    ax.text(FY1[0]+300, FY1[1]+80, 'FY1(17800,0)', fontsize=9)
    ax.scatter(fake_target[0], fake_target[1], c='black', s=80, label='假目标', zorder=5)
    ax.text(fake_target[0]+300, fake_target[1]-120, '假目标(0,0)', fontsize=9)
    ax.scatter(real_target[0], real_target[1], c='green', s=80, label='真目标', zorder=5)
    ax.text(real_target[0]+300, real_target[1]+80, '真目标(0,200)', fontsize=9)
    # 飞行方向（红色虚线）
    ax.plot([M1[0], fake_target[0]], [M1[1], fake_target[1]], 'r--', alpha=0.6, label='导弹飞行方向')
    ax.set_xlabel('X 坐标 (m)', fontsize=11); ax.set_ylabel('Y 坐标 (m)', fontsize=11)

```

```

ax.set_title('图 5 导弹 M1 与无人机 FY1 初始位置水平投影图', fontsize=12, pad=10)
ax.set_xlim(-1000, 21000); ax.set_ylim(-500, 500)
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.08), ncol=5, fontsize=9, frameon=False)
plt.tight_layout()
plt.savefig('fig5_initial_positions.png', dpi=300, bbox_inches='tight')
plt.close()

```

===== 图 6: 基准场景遮蔽状态随时间变化曲线

```

def draw_fig6():
    t_start = 7.62; t_end = 14.32; t_total = 20
    t = np.linspace(0, t_total, 1000)
    shield = np.zeros_like(t)
    shield[(t >= t_start) & (t <= t_end)] = 1
    fig, ax = plt.subplots(figsize=(12, 5.5))
    # 阶跃线（深蓝色）
    ax.plot(t, shield, color='#1f77b4', linewidth=2)
    # 有效遮蔽区域填充（淡蓝色）
    ax.fill_between(t, 0, shield, where=(shield == 1), color='#aec7e8', alpha=0.7)
    # 遮蔽起始（红色虚线）、结束（绿色虚线）
    ax.axvline(x=t_start, color='red', linestyle='--', linewidth=1.5,
               label=f'遮蔽起始 ({t_start:.2f}s)')
    ax.axvline(x=t_end, color='green', linestyle='--', linewidth=1.5,
               label=f'遮蔽结束 ({t_end:.2f}s)')
    ax.set_xlabel('时间 $t$ (s)', fontsize=12)
    ax.set_ylabel('遮蔽状态', fontsize=12)
    ax.set_yticks([0, 1]); ax.set_yticklabels(['无遮蔽', '有效遮蔽'], fontsize=11)
    # 横轴刻度: 0、2.5、5.0、7.5、10.0、12.5、15.0、17.5、20.0
    ax.set_xticks([0, 2.5, 5.0, 7.5, 10.0, 12.5, 15.0, 17.5, 20.0])
    ax.set_ylim(-0.1, 1.15)
    ax.set_title('图 6 基准场景遮蔽状态随时间变化曲线', fontsize=13, pad=15)
    ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.12), ncol=2, fontsize=11, frameon=False)
    plt.tight_layout()
    plt.savefig('fig6_baseline_curve.png', dpi=300, bbox_inches='tight')
    plt.close()

```

===== 图 7：无人机最优飞行方向范围分析示意图

```
def draw_fig7():  
    fig, ax = plt.subplots(figsize=(10, 7))  
    FY1 = np.array([17800, 0]); M1 = np.array([20000, 0])  
    # 绘制点位  
    ax.scatter(FY1[0], FY1[1], c='blue', s=100, label='无人机 FY1', zorder=5)  
    ax.text(FY1[0]+200, FY1[1]+200, 'FY1(17800,0)', fontsize=9)  
    ax.scatter(M1[0], M1[1], c='red', s=100, label='导弹 M1', zorder=5)  
    ax.text(M1[0]+200, M1[1]+200, 'M1(20000,0)', fontsize=9)  
    # 真目标方向标注（箭头指向左方）  
    ax.annotate('真目标(0,200)', xy=(12000, 200), xytext=(13500, 800),  
               fontsize=9, ha='center',  
               arrowprops=dict(arrowstyle='->', color='green', linewidth=1.5))  
    # 最优方向范围（144°~216°，橙色虚线扇形）  
    angle_min = np.deg2rad(144); angle_max = np.deg2rad(216)  
    radius = 5000  
    angles = np.linspace(angle_min, angle_max, 50)  
    ax.plot(FY1[0]+radius*np.cos(angles), FY1[1]+radius*np.sin(angles),  
           'orange', linestyle='--', linewidth=2, label='最优方向范围')  
    ax.plot([FY1[0], FY1[0]+radius*np.cos(angle_min)],  
           [FY1[1], FY1[1]+radius*np.sin(angle_min)], 'orange', linestyle='--')  
    ax.plot([FY1[0], FY1[0]+radius*np.cos(angle_max)],  
           [FY1[1], FY1[1]+radius*np.sin(angle_max)], 'orange', linestyle='--')  
    # 朝向假目标基准方向（红色实线）  
    ax.plot([FY1[0], FY1[0]-radius], [FY1[1], FY1[1]], 'r-', linewidth=1.5, label='朝向假目标方向')  
    ax.set_xlabel('X 坐标 (m)', fontsize=11); ax.set_ylabel('Y 坐标 (m)', fontsize=11)  
    ax.set_title('图 7 无人机最优飞行方向范围分析示意图', fontsize=12, pad=10)  
    ax.set_xlim(12000, 21000); ax.set_ylim(-3000, 3000)  
    ax.set_aspect('equal')  
    ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.06), ncol=4, fontsize=9, frameon=False)  
    plt.tight_layout()  
    plt.savefig('fig7_direction_range.png', dpi=300, bbox_inches='tight')  
    plt.close()
```

===== 图 8: L-BFGS-B 梯度优化迭代收敛曲线

```
def draw_fig8():
    np.random.seed(42)
    iterations = np.arange(0, 35)
    shield_time = 15 + (19.87 - 15) * (1 - np.exp(-iterations / 8))
    shield_time += np.random.normal(0, 0.05, size=len(iterations))
    shield_time = np.maximum.accumulate(shield_time)
    fig, ax = plt.subplots(figsize=(10, 5.5))
    # 收敛曲线（蓝色实线带圆点）
    ax.plot(iterations, shield_time, 'b-', linewidth=2, marker='o', markersize=3)
    # 最优值（红色虚线）
    ax.axhline(y=19.87, color='red', linestyle='--', linewidth=1.5, label='最优值 19.87s')
    ax.set_xlabel('迭代次数', fontsize=11); ax.set_ylabel('有效遮蔽时长 (s)', fontsize=11)
    ax.set_title('图 8 L-BFGS-B 梯度优化迭代收敛曲线', fontsize=12, pad=10)
    ax.set_xlim(0, 35); ax.set_ylim(14.5, 20.5)
    ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.1), fontsize=10, frameon=False)
    plt.tight_layout()
    plt.savefig('fig8_convergence_curve.png', dpi=300, bbox_inches='tight')
    plt.close()
```

===== 图 9: 多枚干扰弹遮蔽时间重叠效应示意图

```
def draw_fig9():
    fig, ax = plt.subplots(figsize=(10, 5))
    t1s, t1e = 2, 15; t2s, t2e = 12, 25; y1, y2 = 1.2, 0.6
    # 第 1 枚弹（蓝色）
    ax.hlines(y=y1, xmin=t1s, xmax=t1e, color='#1f77b4', linewidth=8, label='第 1 枚弹遮蔽区间')
    # 第 2 枚弹（橙色）
    ax.hlines(y=y2, xmin=t2s, xmax=t2e, color='#ff7f0e', linewidth=8, label='第 2 枚弹遮蔽区间')
    # 重叠区域（红色半透明）
    overlap_s = max(t1s, t2s); overlap_e = min(t1e, t2e)
    ax.axvspan(overlap_s, overlap_e, color='red', alpha=0.2, label='重叠区域')
    ax.text((overlap_s+overlap_e)/2, 0.9, '重叠部分', ha='center', fontsize=11, color='darkred')
```



```

ax.set_xlabel('时间 $t$ (s)', fontsize=11)
ax.set_yticks([y2, y1]); ax.set_yticklabels(['第 2 枚弹', '第 1 枚弹'], fontsize=10)
ax.set_title('图 9 多枚干扰弹遮蔽时间重叠效应示意图', fontsize=12, pad=10)
ax.set_ylim(0, 1.8); ax.set_xlim(0, 27)
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.1), ncol=3, fontsize=9, frameon=False)
plt.tight_layout()
plt.savefig('fig9_overlap_effect.png', dpi=300, bbox_inches='tight')
plt.close()

```

===== 图 10: 多枚干扰弹协同遮蔽效应示意图 =====

```

def draw_fig10():
    fig, ax = plt.subplots(figsize=(8, 7.5))
    r_T = 7; r_C = 6
    # 目标截面（绿色边框）
    ax.add_patch(plt.Circle((0, 0), r_T, color='green', fill=False, linewidth=2, label='目标截面'))
    # 云团 1（浅灰色）
    ax.add_patch(plt.Circle((-5, 0), r_C, color='gray', alpha=0.5, label='云团 1'))
    # 云团 2（浅蓝色）
    ax.add_patch(plt.Circle((5, 0), r_C, color='lightblue', alpha=0.5, label='云团 2'))
    ax.text(-5, r_C+0.8, '单枚遮蔽不全', ha='center', fontsize=10, color='gray')
    ax.text(5, r_C+0.8, '单枚遮蔽不全', ha='center', fontsize=10, color='blue')
    ax.text(0, -r_T-2, '协同完全遮蔽', ha='center', fontsize=11, color='darkgreen')
    ax.set_aspect('equal')
    ax.set_xlabel('X', fontsize=11); ax.set_ylabel('Y', fontsize=11)
    ax.set_title('图 10 多枚干扰弹协同遮蔽效应示意图', fontsize=12, pad=10)
    ax.set_xlim(-11, 11); ax.set_ylim(-11, 11)
    ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.06), ncol=3, fontsize=9, frameon=False)
    plt.tight_layout()
    plt.savefig('fig10_cooperative_effect.png', dpi=300, bbox_inches='tight')
    plt.close()

```

===== 图 11: 3 枚干扰弹遮蔽时段衔接示意图 =====

```

def draw_fig11():
    fig, ax = plt.subplots(figsize=(12, 5.5))
    t1_s, t1_e = 2.0, 21.62; t2_s, t2_e = 14.0, 27.85; t3_s, t3_e = 25.0, 32.91

```

```

y_pos = [3, 2, 1]; colors = ['#1f77b4', '#ff7f0e', '#2ca02c']
labels = ['第 1 枚弹', '第 2 枚弹', '第 3 枚弹']
for i, (s, e) in enumerate(zip([t1_s, t2_s, t3_s], [t1_e, t2_e, t3_e])):
    ax.hlines(y=y_pos[i], xmin=s, xmax=e, color=colors[i], linewidth=6, label=labels[i])
# 联合遮蔽（紫色）
ax.hlines(y=0, xmin=t1_s, xmax=t3_e, color='purple', linewidth=8, label='联合有效遮蔽')
ax.set_xlabel('飞行时间 $t$ (s)', fontsize=11)
ax.set_yticks([0, 1, 2, 3])
ax.set_yticklabels(['联合有效遮蔽', '第 3 枚弹', '第 2 枚弹', '第 1 枚弹'], fontsize=10)
ax.set_title('图 11 3 枚干扰弹遮蔽时段衔接示意图', fontsize=12, pad=10)
ax.set_xlim(0, 40); ax.set_ylim(-0.5, 3.8)
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.08), ncol=4, fontsize=9, frameon=False)
plt.tight_layout()
plt.savefig('fig11_3bombs_sequence.png', dpi=300, bbox_inches='tight')
plt.close()

```

===== 图 12: 3 架无人机初始位置水平投影分布图
=====

```

def draw_fig12():
    fig, ax = plt.subplots(figsize=(10, 7))
    M1 = np.array([20000, 0]); fake_target = np.array([0, 0])
    uavs = {'FY1': (17800, 0), 'FY2': (12000, 1400), 'FY3': (6000, -3000)}
    # M1 红色
    ax.scatter(M1[0], M1[1], c='red', s=120, label='导弹 M1', zorder=5)
    ax.text(M1[0]+200, M1[1]+200, 'M1(20000,0)', fontsize=9)
    # FY1 蓝色、FY2 橙色、FY3 绿色
    colors = ['blue', 'orange', 'green']
    for i, (name, pos) in enumerate(uavs.items()):
        ax.scatter(pos[0], pos[1], c=colors[i], s=120, label=name, zorder=5)
        ax.text(pos[0]+200, pos[1]+200, f'{name}({pos[0]}, {pos[1]})', fontsize=9)
    # 假目标黑色
    ax.scatter(fake_target[0], fake_target[1], c='black', s=80, label='假目标', zorder=5)
    ax.text(fake_target[0]+200, fake_target[1]-300, '假目标(0,0)', fontsize=9)
    ax.plot([20000, 0], [0, 0], 'k--', alpha=0.5)
    ax.set_xlabel('X 坐标 (m)', fontsize=11); ax.set_ylabel('Y 坐标 (m)', fontsize=11)

```

```

ax.set_title('图 12 3 架无人机初始位置水平投影分布图', fontsize=12, pad=10)
ax.set_xlim(-1000, 21000); ax.set_ylim(-4000, 2000)
ax.set_aspect('equal')
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.06), ncol=5, fontsize=9, frameon=False)
plt.tight_layout()
plt.savefig('fig12_3uav_positions.png', dpi=300, bbox_inches='tight')
plt.close()

```

===== 图 13: 3 机协同投放空间位置示意图 (3D)

```

def draw_fig13():
    fig = plt.figure(figsize=(10, 7.5))
    ax = fig.add_subplot(111, projection='3d')
    # 导弹飞行轨迹 (红色实线)
    t = np.linspace(0, 1, 100)
    ax.plot(20000*(1-t), np.zeros_like(t), 2000*(1-t),
            'r-', linewidth=2, label='导弹飞行轨迹')
    # 远/中/近三段云团
    cloud_far = np.array([17000, 0, 1700]) # 远段 (灰色, FY1)
    cloud_mid = np.array([12000, 200, 1200]) # 中段 (蓝色, FY2)
    cloud_near = np.array([6000, -100, 600]) # 近段 (绿色, FY3)
    r_C = 200
    cloud_info = [(cloud_far, 'gray', '远段云团(FY1)'),
                  (cloud_mid, 'blue', '中段云团(FY2)'),
                  (cloud_near, 'green', '近段云团(FY3)')]
    for center, color, label in cloud_info:
        u, v = np.mgrid[0:2*np.pi:20j, 0:np.pi:10j]
        ax.plot_surface(center[0]+r_C*np.cos(u)*np.sin(v),
                        center[1]+r_C*np.sin(u)*np.sin(v),
                        center[2]+r_C*np.cos(v),
                        color=color, alpha=0.5)
        ax.text(center[0], center[1], center[2]+r_C+100, label, fontsize=9, ha='center')
    # 真目标
    ax.scatter(0, 200, 0, c='green', s=100, label='真目标')
    ax.set_xlabel('X 轴 (m)', fontsize=10)

```

```

ax.set_ylabel('Y 轴 (m)', fontsize=10)
ax.set_zlabel('Z 轴 (m)', fontsize=10)
ax.set_title('图 13 3 机协同投放空间位置示意图', fontsize=12, pad=15)
# 视角: 俯仰 20°、方位-55°
ax.view_init(elev=20, azim=-55)
ax.legend(loc='upper left', fontsize=9, bbox_to_anchor=(0, 0.95))
plt.tight_layout()
plt.savefig('fig13_3uav_cooperative.png', dpi=300, bbox_inches='tight')
plt.close()

```

===== 图 14: 多机多目标分层优化框架图 =====

```

def draw_fig14():
    fig, ax = plt.subplots(figsize=(8, 6.5))
    ax.axis('off')
    # 上层: 目标分配层 (浅灰色填充)
    upper = plt.Rectangle((0.2, 0.6), 0.6, 0.3, facecolor='#f0f0f0', edgecolor='black', linewidth=2)
    ax.add_patch(upper)
    ax.text(0.5, 0.75, '目标分配层\n (遗传算法)', ha='center', va='center', fontsize=12)
    # 下层: 策略优化层 (浅蓝色填充)
    lower = plt.Rectangle((0.2, 0.1), 0.6, 0.3, facecolor='#e0f0ff', edgecolor='black', linewidth=2)
    ax.add_patch(lower)
    ax.text(0.5, 0.25, '策略优化层\n (协同优化算法)', ha='center', va='center', fontsize=12)
    # 中间向下箭头
    ax.annotate("", xy=(0.5, 0.6), xytext=(0.5, 0.4),
                arrowprops=dict(arrowstyle='->', linewidth=2, color='black'))
    # 左侧输入
    ax.text(0.08, 0.75, '无人机\n 导弹', ha='center', va='center', fontsize=10)
    ax.annotate("", xy=(0.2, 0.75), xytext=(0.13, 0.75),
                arrowprops=dict(arrowstyle='->', linewidth=1.5, color='black'))
    # 右侧输出
    ax.text(0.92, 0.25, '最优投放\n 策略', ha='center', va='center', fontsize=10)
    ax.annotate("", xy=(0.87, 0.25), xytext=(0.8, 0.25),
                arrowprops=dict(arrowstyle='->', linewidth=1.5, color='black'))
    ax.set_title('图 14 多机多目标分层优化框架图', fontsize=12, pad=10)
    plt.tight_layout()

```

```
plt.savefig('fig14_hierarchical_framework.png', dpi=300, bbox_inches='tight')
plt.close()
```

```
# ===== 图 15：关键参数变化对遮蔽时长的影响对比 =====
```

```
def draw_fig15():
    x = np.arange(5)
    labels = ['-20%', '-10%', '基准', '+10%', '+20%']
    width = 0.25
    # 精准数据
    rc_change = [-22, -11, 0, 12, 23] # 云团半径：正相关
    vc_change = [12, 6, 0, -7, -13] # 下沉速度：负相关
    vm_change = [15, 7, 0, -8, -15] # 导弹速度：负相关
    fig, ax = plt.subplots(figsize=(10, 6))
    # 云团半径蓝色、下沉速度橙色、导弹速度绿色
    ax.bar(x - width, rc_change, width, label='云团半径 $r_C$', color='#1f77b4')
    ax.bar(x, vc_change, width, label='下沉速度 $v_C$', color='#ff7f0e')
    ax.bar(x + width, vm_change, width, label='导弹速度 $v_M$', color='#2ca02c')
    ax.axhline(y=0, color='black', linewidth=1)
    ax.set_xticks(x); ax.set_xticklabels(labels, fontsize=11)
    ax.set_ylabel('遮蔽时长变化率 (%)', fontsize=11)
    ax.set_xlabel('参数变化幅度', fontsize=11)
    ax.set_title('图 15 关键参数变化对遮蔽时长的影响对比', fontsize=12, pad=10)
    ax.legend(fontsize=10, loc='upper center', bbox_to_anchor=(0.5, -0.08), ncol=3, frameon=False)
    ax.set_ylim(-30, 30)
    plt.tight_layout()
    plt.savefig('fig15_sensitivity_analysis.png', dpi=300, bbox_inches='tight')
    plt.close()
```

```
# ===== 批量生成 =====
```

```
if __name__ == '__main__':
    print("正在生成 15 张论文插图...")
    draw_fig1(); print("图 1 完成")
    draw_fig2(); print("图 2 完成")
    draw_fig3(); print("图 3 完成")
```

```
draw_fig4(); print("图 4 完成")
draw_fig5(); print("图 5 完成")
draw_fig6(); print("图 6 完成")
draw_fig7(); print("图 7 完成")
draw_fig8(); print("图 8 完成")
draw_fig9(); print("图 9 完成")
draw_fig10(); print("图 10 完成")
draw_fig11(); print("图 11 完成")
draw_fig12(); print("图 12 完成")
draw_fig13(); print("图 13 完成")
draw_fig14(); print("图 14 完成")
draw_fig15(); print("图 15 完成")
print("\n 全部 15 张图生成完毕！")
```